PIPELINING

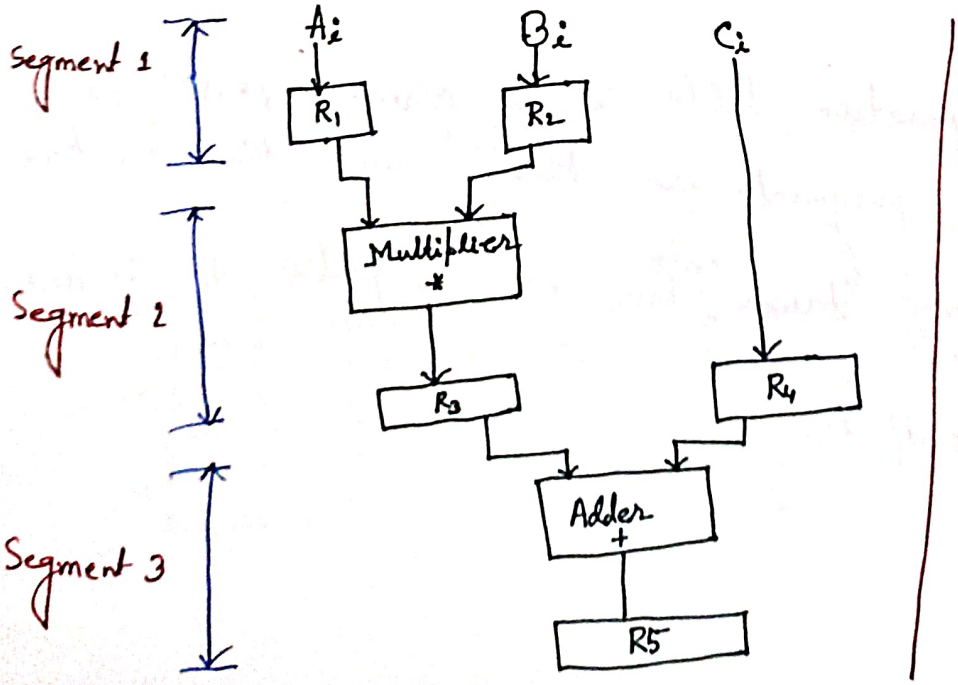
* This is a special kind of technique used in advanced microprocessors where a sequential process is decomposed into sub-operations and each suboperations are performed/executed in a segment which operates concurrently with all other segments.

To understand this you can assume ~~your daily routine~~ you live and your two friends in a room having one bathroom, one kitchen and one dressing table.

If each person ~~was~~ performs one work at a time, ~~or~~ then there will be difference in performing concurrently and performing separately.

* Example for pipelining ⟶

$A_i * B_i + C_i$ for $i = 1, 2, \ldots\ldots 7$.



$R_1 \leftarrow A_i$, $R_2 \leftarrow B_i$ Input A_i and B_i

$R_3 \leftarrow R_1 * R_2$, $R_4 \leftarrow C_i$ Multiply and input C_i

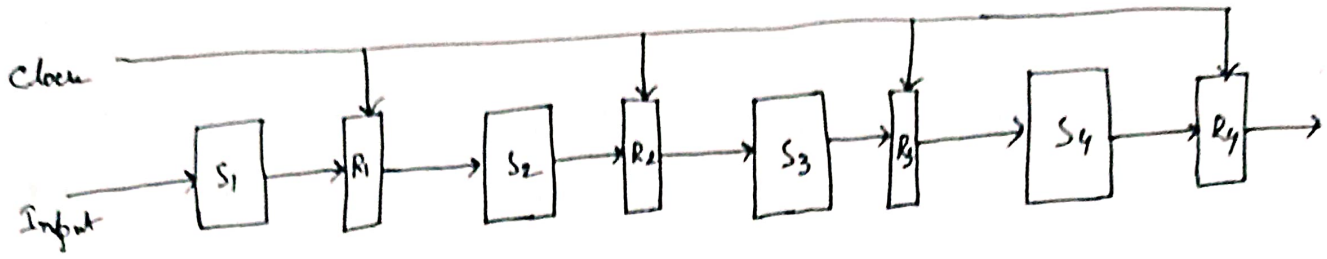
$R5 \leftarrow R3 + R4$ Add C_i to product.

* Operations in each stage →

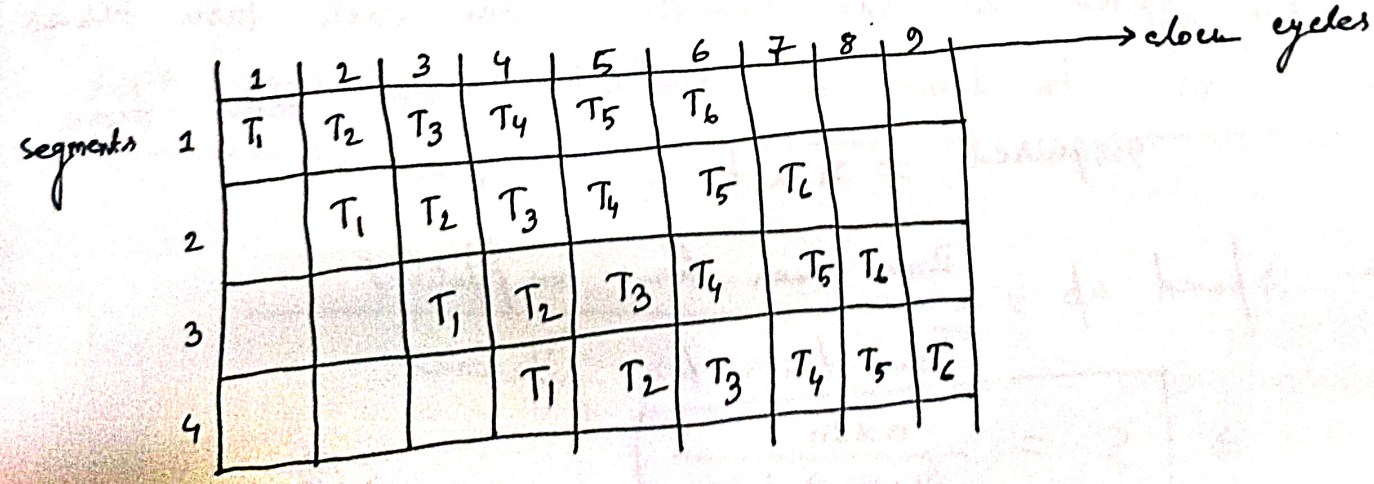
clock pulse	Segment 1		Segment 2		Segment 3
	R_1	R_2	R_3	R_4	R_5
1.	A_1	B_1	....	....	....
2.	A_2	B_2	$A_1 * B_1$	C_1	....
3.	A_3	B_3	$A_2 * B_2$	C_2	$A_1 * B_1 + C_1$
4.	A_4	B_4	$A_3 * B_3$	C_3	$A_2 * B_2 + C_2$
5.	A_5	B_5	$A_4 * B_4$	C_4	$A_3 * B_3 + C_3$
6.	A_6	B_6	$A_5 * B_5$	C_5	$A_4 * B_4 + C_4$
7.	A_7	B_7	$A_6 * B_6$	C_6	$A_5 * B_5 + C_5$
8.	....	....	$A_7 * B_7$	C_7	$A_6 * B_6 + C_6$
9.	....	....	....	....	$A_7 * B_7 + C_7$

From the operation table, it is obvious that no matter how many segments are there, once the pipeline is full, it only takes one clock pulse to produce an output.

* Four segment pipeline → For any four segment pipeline,



In this case, for the first clock cycle task T_1 is executed in segment 1 while other segments are waiting for the inputs. In the second clock segment 1 handles next task T_2 while segment 2 handles task T_1 and others waiting for inputs. This process will continue till fourth clock pulse. At that time T_1 task is reached to segment 4. After this each clock pulse will produce an output. This can be understood from the space-time diagram, where, along x-axis clock cycles are plotted and segments are mentioned along y-axis. The space-time diagram is given below for total six tasks (T_1 to T_6).



* A n ... to process a task. The same with of

* **Speed-up** → This parameter of a pipelining system determines how efficient the pipelined system compared to its non-pipelined structure.

Ans consider a system of k-segments and clock cycle is t_p is used to perform total n tasks.

The first task T_1 will take $k \times t_p$ time to complete.

The second task T_2 will take $(k+1) \times t_p$ time to complete.

$T_3 \longrightarrow (k+2) \times t_p$

Thus, one can, ~~say remaining (n-1) task first~~ task needs $k \times t_p$ time and remaining $(n-1)$ task will take $(n-1) \times t_p$ time.

Thus, total time taken for the pipelined system

$$k \times t_p + (n-1) \times t_p$$
$$= (k+n-1) \times t_p \longrightarrow (1)$$

* If the system is non pipelined and each task takes t_n time to complete, then total time required $= n \times t_n \longrightarrow (2)$

Thus, speed up $= \dfrac{\text{Time taken for non pipelined}}{\text{Time taken for pipelined}}$

$$\Rightarrow \boxed{S = \frac{n \times t_n}{(k+n-1)\, t_p}} \longrightarrow (3).$$

* A non-pipelined system takes 50 ns to process a task. The same task can be processed in a six segment pipeline with a clock cycle of 10 ns. Determine the speed up of the pipeline system for 100 task.

Ans: Here, $t_n = 50$ ns.

$t_p = 10$ ns.

$k = 6$

and $n = 100$

To complete 100 tasks,

non-pipeline system needs $100 \times t_n$ time.

and pipe-lined system needs, $(k + n - 1) \times t_p$ time.

Thus, speed up, $S = \dfrac{100 \times t_n}{(k+n-1)\, t_p}$

$\Rightarrow S = \dfrac{100 \times 50}{(6+100-1) \times 10}$

$\Rightarrow S = \dfrac{100 \times 50}{105 \times 10} = 4.76$

*** Remember speed up is a unitless quantity.

Pipeline

* For any pipe line process, the cycle time is determined as, T_p

$$T_p = \max(T_1, T_2, \ldots, T_k)$$

↳ k stages.

If there are latency in 0 different registers

then, $\text{Latency} = \sum_{i=1}^{k} T_i$

i ⇒ stage number

If there are n number of tasks,

$$\text{Total time} = (k+n-1)\, T_p .$$

Thus, $\text{throughput} = \dfrac{\text{Total task number}}{\text{Total time}}$

$$= \frac{n}{(k+n-1)\, T_p}$$

If $n >> k$, then $k+n-1 \approx n$.

Thus, $\text{throughput} \approx 1/T_p$.

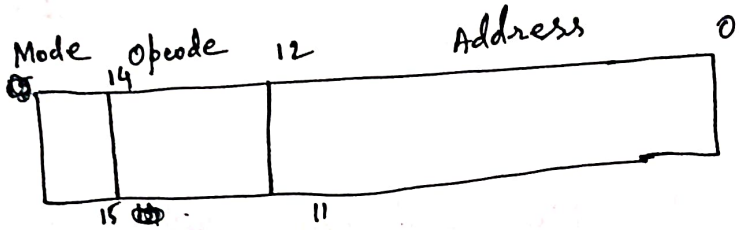
Example → Suppose, there are four stages as, T_1 as, 150, 120, 160 and 140 ns and 1000 tasks having stage delay 5 ns (Register) delay.

Then time, $T_1 = 150 + 5 = 155$ ns
$T_2 = 120 + 5 = 125$ ns
$T_3 = 160 + 5 = 165$ ns
$T_4 = 140 + 5 = 145$ ns.

INSTRUCTION FORMAT

The instruction of any computer has three very basic parts,

1. Mode → Determines the mode
2. Opcode → Determine operation
3. Address → Provide the address.

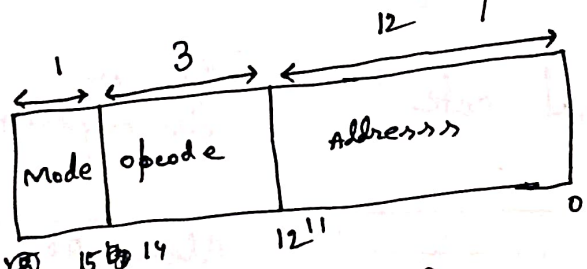


Basically, there are three types of operation in a computer,

i) Memory-Reference → 0–11 bits represents address.

I (15)
- 0 for direct addressing mode
- 1 for indirect addressing mode.

Opcode (12–14) determines the operation.



Symbol	Hexadecimal code: Direct (I=0) $D_{15}D_{14}D_{13}D_{12}$	Hexadecimal code: Indirect (I=1) $D_{15}D_{14}D_{13}D_{12}$	Description
AND	0 XXX 0000 XXX	8 XXX 1000 XXX	AND memory word to AC
ADD	1 XXX 0001 XXX	9 XXX 1001 XXX	ADD memory word to AC
LDA	2 XXX 0010 XXX	A XXX 1010 XXX	LOAD memory word to AC
STA	3 XXX 0011 XXX	B XXX 1011 XXX	STORE content of AC to memory.

Each X is four bit (Hexadecimal) total → 16 bits.

* How instructions are executed ⟶

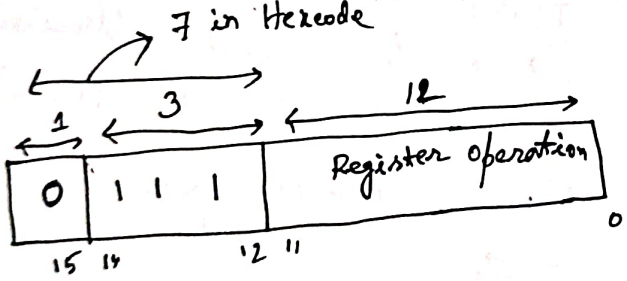
Symbol	Hexadecimal code Direct (I=0)	Indirect (I=1)	Description
BUN ⟶	4 XXX 0100 XXX	C XXX ⟶ 1100 XXX	Branch unconditionally
BSA ⟶	5 XXX 0101 XXX	D XXX ⟶ 1101 XXX	Branch and save return address.
ISZ ⟶	6 XXX 0110 XXX	E XXX ⟶ 1110 XXX	Increment and skip if zero.

ii) Register reference → This instruction specifies the operation on accumulator (AC).

0 - 11 bits determine the operation.

12 - 14, is 111

15th bit is 0.



Symbol	Hexadecimal code	Description
CLA ⟶	7800 ⟶	clear AC
CLE ⟶	7400 ⟶	clear E.
CMA ⟶	7200 ⟶	complement AC
CME ⟶	7100 ⟶	complement E.
CIR ⟶	7080 ⟶	circulate AC and E Right.
CIL ⟶	7040 ⟶	circulate AC and E Left.
INC ⟶	7020 ⟶	Increment AC.
SPA ⟶	7010 ⟶	Skip next instruction if AC is positive.

* How instructions are executed ⟶

PC contains the address of next instruction

① PC transfer the address of instruction to AR (T_0). At that time sc is also cleared (SC = 0).

$$AR \leftarrow PC \quad , \quad SC \leftarrow 0 \qquad (T = T_0)$$

Fetch

In the next cycle (T_1), the data from the memory location having address at AR will be fetched to the MDR and transferred to IR. PC will be incremented to fetch the next instruction.

$$IR \leftarrow M[AR] \quad , \quad PC \leftarrow PC + 1 \qquad (T = T_1)$$

Decode

In the next clock cycle (T_2), the opcode of IR (12, 13 and 14 bits) are decoded to know the type of operation.

$$D_0 \ldots\ldots D_7 \leftarrow \text{Decode IR}(12-14), \qquad (T = T_2)$$
$$AR \leftarrow IR(0-11)$$
$$I \leftarrow IR(15)$$

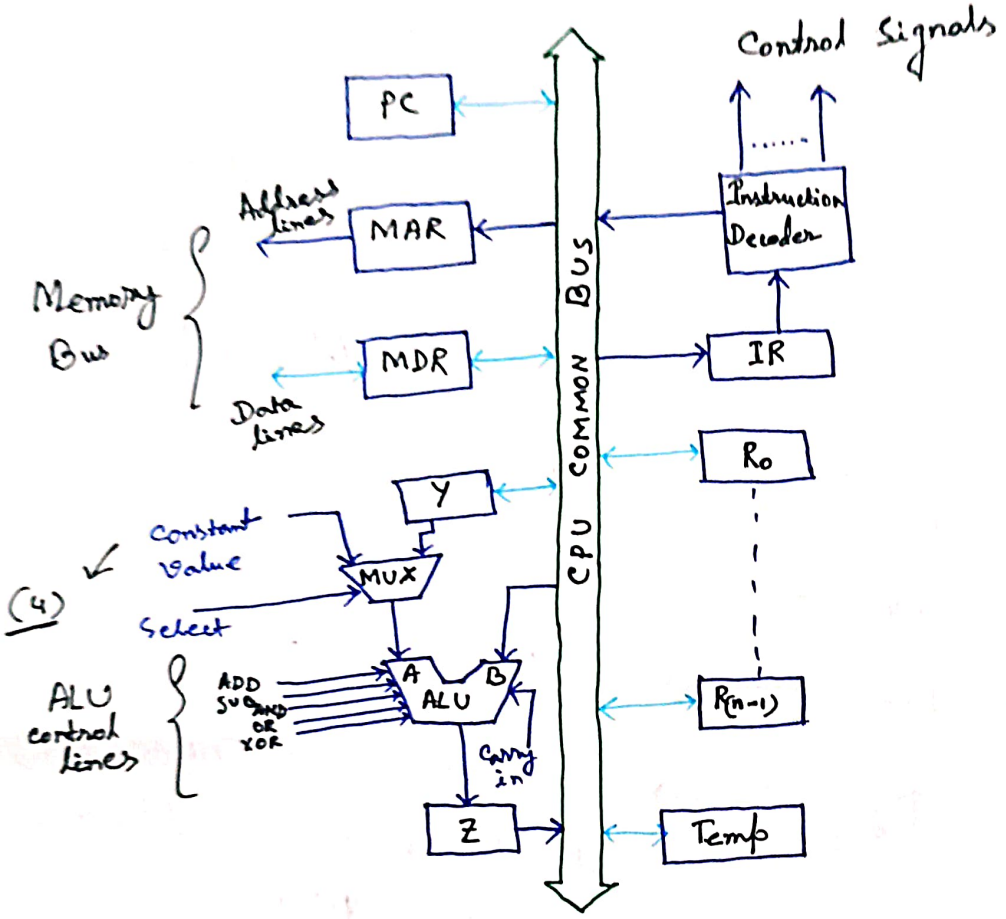
Execution

In the next clock cycle the operation is executed based on the previously determined value of ② opcode and mode.

$D_7' I\, T_3$: $AR \leftarrow M[AR]$
$D_7' I'\, T_3$: D_0 nothing
$D_7 I\, T_3$: I/O instruction execution
$D_7 I'\, T_3$: Register reference instruction execution

$(T = T_3)$

* Single Bus Organization of a Processor ⟶ In this type of computer organization ALU and all processor registers are connected to a single common bus.



Here, the registers Y, Z and Temp only used by CPU for temporary storage without any human user. R_0 to $R_{(n-1)}$ registers are general purpose registers. MUX has two input to select, a constant value (normally 4) or output of Y. ALU has two options the input A, depending upon MUX selection. Here the constant value is used to increase the PC, it may be 1 or 4.

Symbol | Hexadecimal code | Description

(5)

* Memory Reference Instruction → Here, $D_7 = 0$ and $I = 0$ for direct and $D_7 = 0$ and $I = 1$ for indirect memory reference. EA is given in AR. The data from memory must must be placed from memory ~~(read)~~ (Read) to a register where they can be operated.

Symbol	operation Decoder	Description
AND →	D_0 →	$AC \leftarrow AC \wedge M[AR]$.
ADD →	D_1 →	$AC \leftarrow AC + M[AR]$, $E \leftarrow Cout$
LDA →	D_2 →	$AC \leftarrow M[AR]$
STA →	D_3 →	$M[AR] \leftarrow AC$
BUN → (Branch unconditional)	D_4 →	$PC \leftarrow AR$
BSA → (Branch and save return address)	D_5 →	$M[AR] \leftarrow PC$, $PC \leftarrow AR+1$
ISZ → (Increment and skip if zero)	D_6 →	$M[AR] \leftarrow M[AR] + 1$, If $M[AR]+1 = 0$ then $PC \leftarrow PC+1$

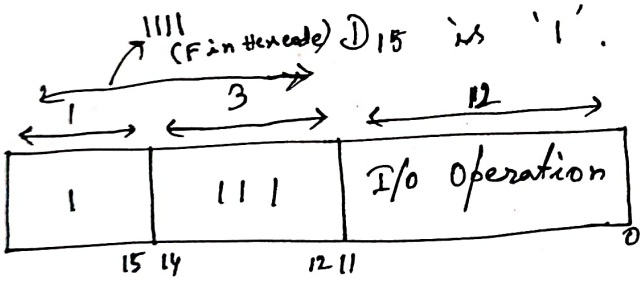
Symbol	Hexadecimal code	Description
SNA	7008	Skip next instruction if AC is negative.
SZA	7004	Skip next instruction if AC is zero.
SZE	7002	Skip next instruction if E is zero.
HLT	7001	Halt computer.

iii) Input-output → This instruction specifies the operation in input-output terminal.

0-11 bit specify the I/O operation

12-14 is 111

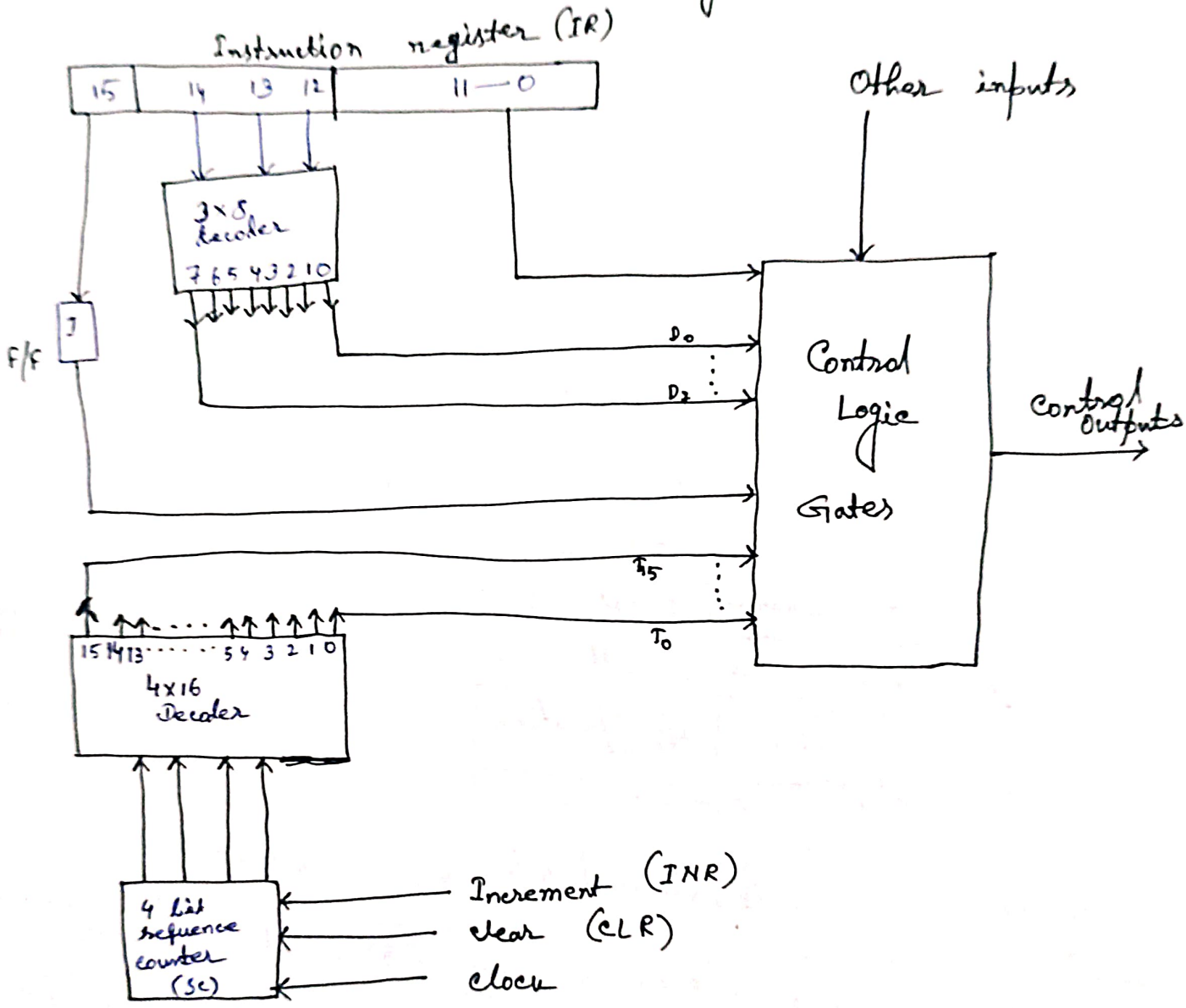
D15 is '1'.



Symbol	Hexadecimal code	Description
INP	F800	Input to AC
OUT	F400	output from AC.
SKI	F200	Skip on input flag.
SKO	F100	Skip on output flag.
ION	F080	Interrupt on.
IOF	F040	Interrupt off.

Total 25 instructions are there.

* Control unit → The control unit of a computer consists of two decoder, a sequence counter and a number of control logic gates.



An instruction read from memory is placed in IR. The opcode (12-14) is decoded by a 3x8 decoder as D_0 to D_7. Bit 15 (mode bit) is transferred to flip-flop 'I'. Bits 0-11 are applied directly to the control logic gates. The sequence counter can count 0-15 in binary. The output of the counter is fed in a 4x16 decoder to generate 16 timing signals (T_0 to T_{15}).

INSTRUCTION CYCLE

* Any program residing in the memory unit of a computer consists of a sequence of instructions. Here, the term 'sequence' plays crucial role to execute the instructions in a perfect order to get the unique outcome. Therefore, the program is executed in the computer by going through a cycle for each instruction. Each cycle may be subdivided into a sequence of subcycles or, phases.

i) Fetch the instruction from memory
ii) Decode the instruction to understand the operation.
iii) Read the effective address for indirect addressing.
iv) Execution of instruction.

The steps repeats untill a HALT instruct (End) is encountered.

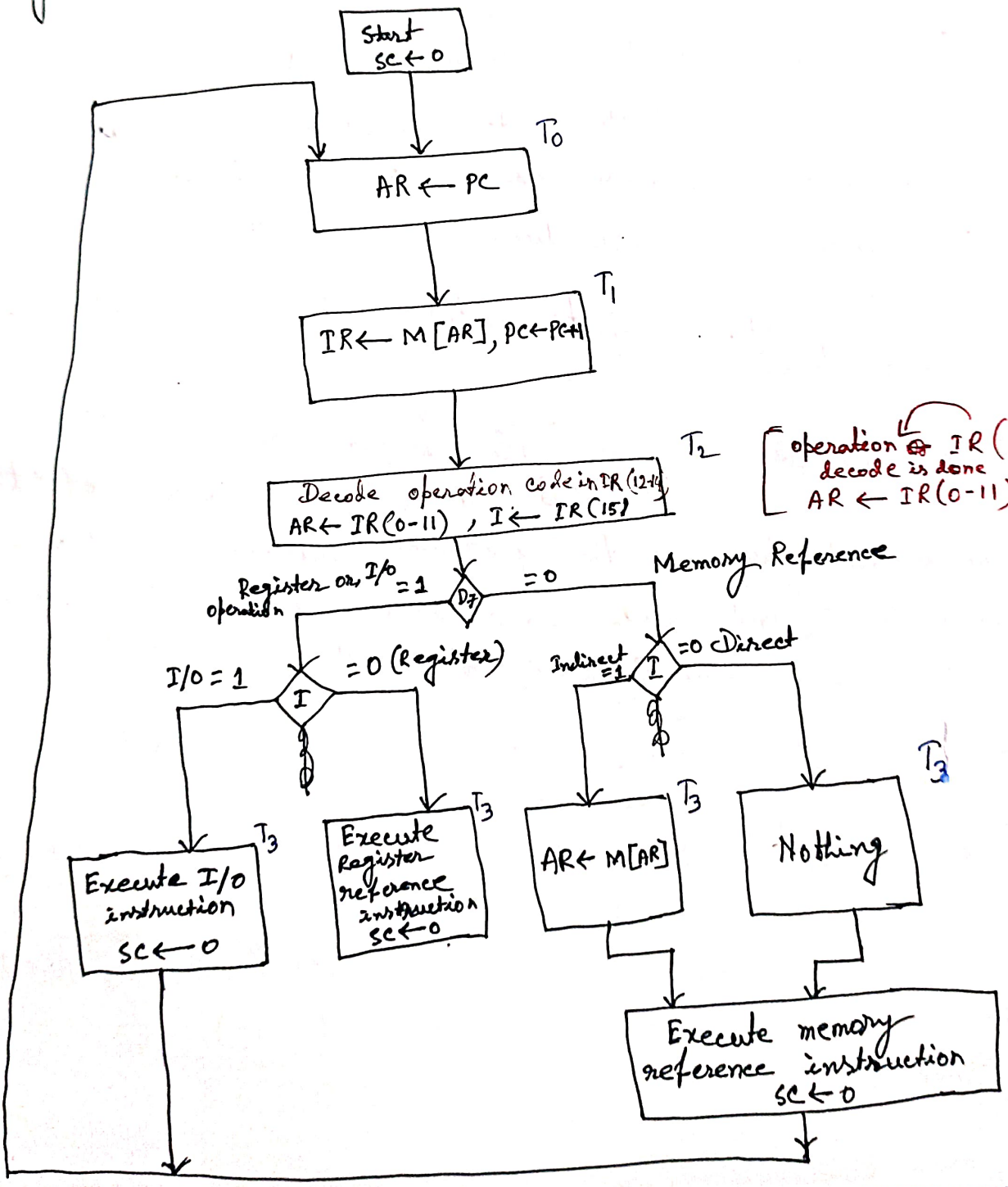
Fetch and Decode → The program counter (PC) is loaded with the address of the first instruction in the program. Sequence counter is cleared to 0, at T_0 and after each clock pulse SC is incremented by 1. The micro-operations are as,

T_0 : AR ← PC

T_1 : IR ← M[AR], PC ← PC + 1

T_2 : $D_0, \ldots\ldots, D_7$ ← Decode IR(12-14), AR ← IR(0-11),
I ← IR(15).

At time T_0, the address from PC is transfered to AR. Then the instruction is read from the address to IR at time T_1. At the same time PC will be increased by 1 to fetch the next instruction. At time T_2, the operation code of IR is decoded. The indirect bit is transfered to I (F/F), address part to AR. During T_3 the control unit determines the type of instruction which is just read. The flowchart is given below to understand the instruction cycle.



operation decode is done
IR(12-14)
AR ← IR(0-11), I ← IR(15)

* Execution of register reference instruction → In this case, $D_7 = 1$ but $I = 0$. The 12 bits of IR (0-11) are used to specify 12 different instructions as,

$D_7 I' T_3 = r$ (common to all register reference instruction).

$IR(i) = B_i$ (IR (0-11) → specifies the instruction).

E ⇒ Extended bit of AC & single bit register used for carry.

CLA →	rB_{11} :	$AC \leftarrow 0$	clear AC (Accumulator).
CLE →	rB_{10} :	$E \leftarrow 0$	clear E.
CMA →	rB_9 :	$AC \leftarrow \overline{AC}$	complement AC.
CME →	rB_8 :	$E \leftarrow \overline{E}$	complement E
CIR →	rB_7 :	$AC \leftarrow shr\ AC$, $AC(15) \leftarrow E$, $E \leftarrow AC(0)$	circulate right.
CIL →	rB_6 :	$AC \leftarrow shl\ AC$, $AC(0) \leftarrow E$, $E \leftarrow AC(15)$	circulate left.
INC →	rB_5 :	$AC \leftarrow AC + 1$	Increment AC.
SPA →	rB_4 :	If $AC(15) = 0$ then $PC \leftarrow PC + 1$	skip if positive.
SNA →	rB_3 :	If $AC(15) = 1$ then $PC \leftarrow PC + 1$	skip if negative.
SZA →	rB_2 :	If $AC = 0$, then $PC \leftarrow PC + 1$	Skip if AC is zero
SZE →	rB_1 :	If $E = 0$, then $PC \leftarrow PC + 1$	Skip if E is zero
HLT →	rB_0 :	$S \leftarrow 0$	Halt.

Now, the value of $D_7 = 1$ if the opcode is 111. The opcode 111 is possible while the instruction is either register or I/O operation. If $D_7 = 0$, then the opcode may have any value in between 000 to 110, which refers to a memory operation. The control unit then search to know the value of I (F/F). If $D_7 = 0$ and $I = 1$, then it is a memory reference, indirect instruction. Then EA (Effective address) is needed and $AR \leftarrow M[AR]$ will be carried out. When $D_7 = 0$ and $I = 0$, then no operation is needed as the EA is given directly in AR. Only the sequence counter needs to be increased ($sc \leftarrow sc+1$). After any instruction is executed sc is cleared to zero ($sc \leftarrow 0$).

All the four operations are symbolized as,

$D_7' I T_3 : AR \leftarrow M[AR]$ ⟶ ($D_7 = 0$, $I = 1$ and T_3 time period).

$D_7' I' T_3 :$ Do Nothing. ⟶ (D_7, $I = 0$ and T_3 time period).

$D_7 I' T_3$: ⟶ ($D_7 = 1$, $I = 0$ and T_3 time period).

$D_7 I' T_3 :$ Register Reference instruction execution.

$D_7 I T_3 :$ I/O Instruction execution. ⟶ ($D_7 = 1$, $I = 1$ and T_3 time period).

* Execution of a complete instruction,

ADD R_1, (R_3)

Ans:

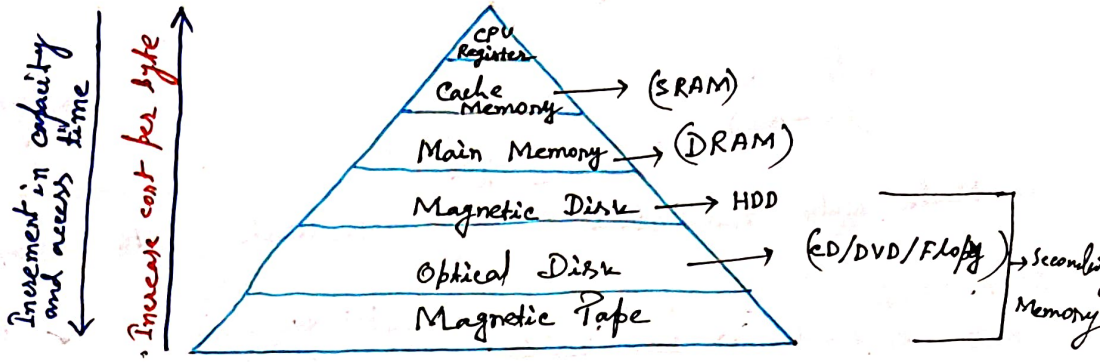
	Time steps	Microoperations	Description
Fetch	T_0	$MAR \leftarrow PC$	Transfer the address of next instruction.
	T_1	$MDR \leftarrow M[MAR]$, $PC \leftarrow PC+1$	Fetch or read the instruction from memory. PC is also incremented to point the next instruction.
	T_2	$IR \leftarrow MDR$	Load instruction from memory to MDR to IR.
Decode	T_3	Decode	Determine the type and actual task of the instruction.
Execution	T_4	$MAR \leftarrow R_3$, $MDR \leftarrow M[MAR]$	Fetch first operand as it is indirect address mode.
	T_5	$R_1 \leftarrow R_1 + MDR$, Updation of flags. End.	Perform the addition and update the flag.
	T_6	Next instruction fetch	Begining of next instruction

* Write the sequence of microoperations a computer system performs to execute the register reference instruction CIR (Circular Shift Right). Assume that the 12-bit address field of a 16 bit IR carries 0000 0000 0111.

Ans:

	Time step	Microoperations	Description
Fetch	T_0	MAR ← PC	Transfer the address of the next instruction.
	T_1	MDR ← M[MAR], PC ← PC+1	Fetch instruction from memory (Read). Increment of PC.
	T_2	IR ← MDR	Load instruct from memory to MDR to IR.
Decode	T_3	Decode	Decode the type of instruction and the instruction task (CIR).
Execute	T_4	$R_{Temp} \leftarrow AC_0$, AC ← AC >> 1, $AC_{15} \leftarrow L$, $L \leftarrow R_{Temp}$, End	Execute circular right shift.
	T_5	Next cycle Fetch.	Begining of next cycle.

* Memory Pyramid →

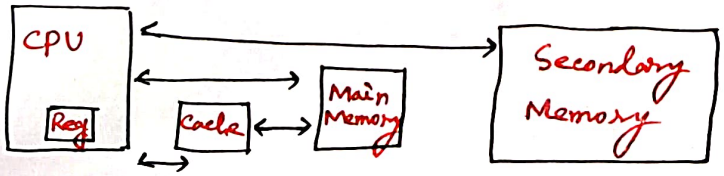


Registers: Small but very fast (Fastest) memory unit located inside cpu. These are used to store most frequently used data and instructions.
The size ranges from 8 to 64 bits (Typically).

Cache Memory: Small but fast memory unit located close to cpu. It stores frequenty used data and instructions of main memory to provide faster operating speed.
The size ranges from 32 kB to 96 MB (Typically).

Main Memory: This memory stores data and instructions that are currently used by cpu and it is volatile. Moderately high cost memory low access time.
The size ranges from 512 MB to some hundred of GBs (Typically).

Secondary Memory: These are less expensive and non volatile memory. These are used to store permanent data for future use. The access time is very high. There are various types of secondary memory like Hard Disk Drive (HDD), SSD, CD, DVD, etc.

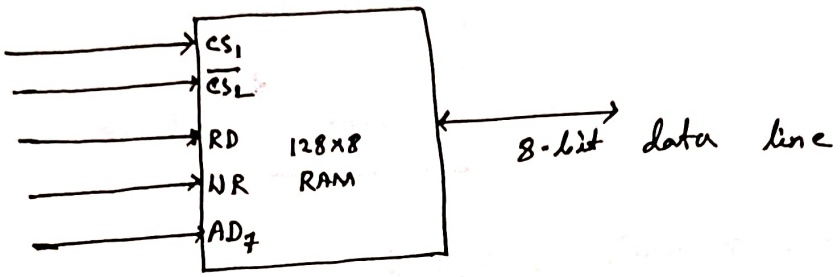


* Difference of Dynamic and Static RAM

Static RAM (SRAM)	Dynamic RAM (DRAM)
1. Made with transistors	1. Made with transistors and capacitors.
2. No refresh is needed. Data is secured as long as the power is ON.	2. Refresh is essential after few milisecon as charge leakage is happened in capacitor.
3. Operating speed is very fast.	3. Operating speed is slower due to refresh cycle.
4. More expensive	4. Less expensive.
5. Mainly used in cpu cache	5. Mainly used in main memory
~~6. High Density~~	
6. Low density (Typically ~~4 to 6 transistors~~ 4-6 transistors), Low capacity.	6. High density (Typically 1 transistor and 1 capacitor), High capacity.

Density → More components per bit.

* **RAM →**



CS_1 ⇒ chip select 1 (Activates the RAM when '1')

$\overline{CS_2}$ ⇒ chip select 2 (Activates the RAM when '0').

RD ⇒ Read

WR ⇒ Write

AD_7 ⇒ 7-bit address lines

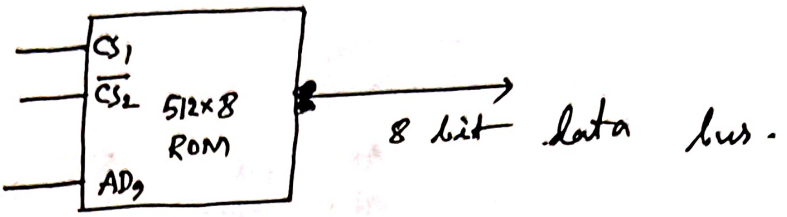
128 × 8 RAM means 128 words of 8 bit each word size.

RAM operation

CS_1	$\overline{CS_2}$	RD	WR	Memory function	State of data bus
0	0	X	X	Inhibit	High impedance
0	1	X	X	Inhibit	High impedance
1	0	0	0	Inhibit	High impedance
1	0	0	1	Write	Input to RAM
1	0	1	0	Read	Output from RAM
1	1	X	X	Inhibit	High impedance

X → Means do not care (Independent of 0 or, 1).

* ROM →



CS_1 ⇒ Chip select 1

$\overline{CS_2}$ ⇒ chip select 2

AD_9 ⇒ Address line (2^9 = 512)

ROM has only Read operation and to activate the chip CS_1 and $\overline{CS_2}$ will be 1 and 0 respectively.

Therefore, while connecting to CPU, CS_1 is connected to RD (Read).

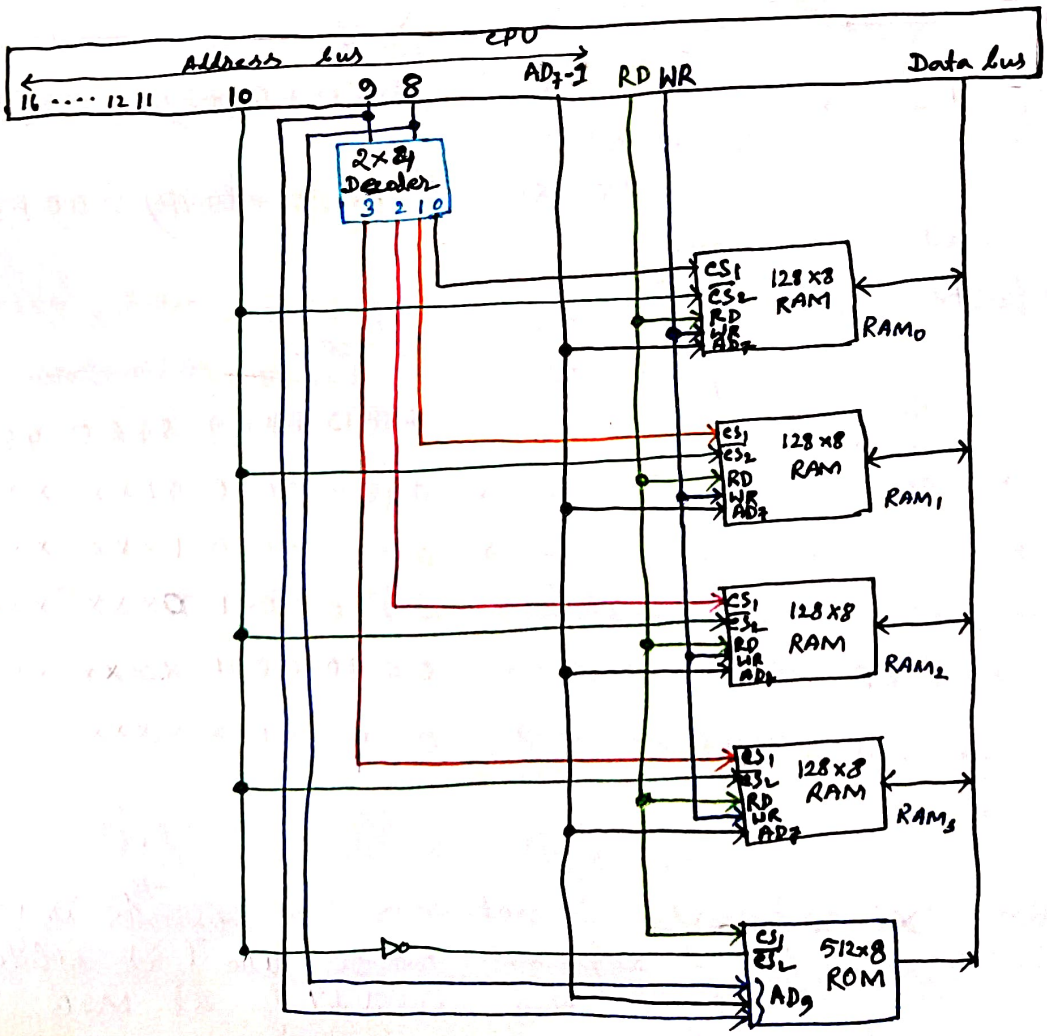
Q1. Suppose you are designing a computer system having RAM and ROM chips of 128×8 and 512×8. The total main memory needs to be 512×8. How many RAM chips are required? Design the whole system and provide the memory address mapping.

Ans: RAM chip size 128×8 → Word size; ↓ Number of words.

Total main memory or, RAM = 512×8

Thus, $\frac{512}{128} = 4$ RAM chips are needed.

The whole system becomes,



Memory address mapping → The given size of each RAM is
$128 \times 8 = 128$ bytes.

Thus, the first RAM (RAM_0) will starts from 0 and ends at 127.

Next RAM will start just after it and so on.

Now, $(127)_{10} = (1111111)_2$

$= (0000\ 0000\ 0111\ 1111)_2$

$= (0\ 0\ 7F)_{16}$.

For your understanding, do not write in examination

Therefore, 007F is the range and each time we just add it to the starting address.

For, RAM_0 → start 0000 end (0000 + 007F) = 007F

RAM_1 → (007F + 1) = 0080 (0080 + 007F) = 00FF

and so on.

The complete map is given below.

Component	Hexadecimal address	16 15 14 13 12 11 10 9 8 7 6 5 4 3 2 1
RAM_0 →	0000 — 007F →	0000 0000 01XX XXXX
RAM_1 →	0080 — 00FF →	0000 0000 1XXX XXXX
RAM_2 →	0100 — 017F →	0000 0001 0XXX XXXX
RAM3 →	0180 — 01FF →	0000 0001 XXXX XXXX
ROM →	0200 — 03FF →	0000 001X XXXX XXXX

***

Here, 'X' represents do not care it is used to represent range. Use it while two 1's are used consecutively at MSB.

*** ROM size 512 → Range 0200 — 01FF.

9 Address ranges

Q2. A computer employs RAM chips of 256 x 8 and ROM chips of 1024 x 8. The computer system needs 2K bytes of RAM and 4K bytes of ROM and 4 interface units each with 4 registers. A memory mapped I/O configuration is used. The two high order bits of the address bus is used as 00 for RAM, 01 for ROM and 10 for interface registers.

a) How many RAM and ROM chips are needed?

b) Draw the memory address map for the whole system.

c) Give the address range in hexadecimal for RAM, ROM and interface register.

d) Provide the size of decoders.

Ans: Given size of each RAM = $\frac{256 \times 8}{\text{Words} \quad \text{Size of word}}$.

Given size of each ROM = $\frac{1024 \times 8}{\text{Words} \quad \text{size of word}}$.

Total RAM = 2K = 2 x 1024

and ROM = 4K = 4 x 1024

a) Thus, number of RAM = $\frac{\text{Total RAM}}{\text{Each size}} = \frac{2 \times 1024}{256} = 8$

and number of ROM = $\frac{\text{Total ROM}}{\text{Each size}} = \frac{4 \times 1024}{1024} = 4$.

b) Now, $256 = 2^8$

Thus, 8 address lines for RAM

and $2^{10} = 1024$,

Thus, 10 address lines for ROM.

Given RAM has MSB lines as 00.

Thus, RAM_0 will start from 0 to 255.

$(255)_{10} \longrightarrow \underset{0}{0000}\ \underset{0}{0000}\ \underset{F}{1111}\ \underset{F}{1111}$ (Range).

ROM has MSB lines as 01.

Thus, ROM_0 will start from $\underset{4}{0100}\ \underset{0}{0000}\ \underset{0}{0000}\ \underset{0}{0000}$

to $\underset{4}{0100}\ \underset{3}{0011}\ \underset{F}{1111}\ \underset{F}{1111}$ (Range)

as 0 – 1023

→ 0000 0011 1111 1111

only for your understanding
Do not write in examination.

Again, Interface registers have MSB 10

Thus, register$_0$ (First) starts from $\underset{8}{1000}\ \underset{0}{0000}\ \underset{0}{0000}\ \underset{0}{0000}$.

Total internal register = $4 \times 4 = 16$.

The complete map is given below,

Component	Address in Hexadecimal		16 15 14 13	12 11 10 9	8 7 6 5	4 3 2 1
RAM_0	0000 – 00FF	→	0000	0000	1XXX	XXXX
RAM_1	0100 – 01FF	→	0000	0001	XXXX	XXXX
RAM_2	0200 – 02FF	→	0000	0010	XXXX	XXXX
RAM_3	0300 – 03FF	→	0000	001X	XXXX	XXXX
RAM_4	0400 – 04FF	→	0000	0100	XXXX	XXXX
RAM_5	0500 – 05FF	→	0000	0101	XXXX	XXXX
RAM_6	0600 – 06FF	→	0000	0110	XXXX	XXXX
RAM_7	0700 – 07FF	→	0000	01XX	XXXX	XXXX
ROM_0	4000 – 43FF	→	0100	0001	XXXX	XXXX
ROM_1	4400 – 47FF	→	0100	0100	XXXX	XXXX
ROM_2	4800 – 4BFF	→	0100	1000	XXXX	XXXX
ROM_3	4C00 – 4FFF	→	0100	1XXX	XXXX	XXXX
Int. Reg$_0$	8000	→	1000	0000	0000	0000
Reg$_1$	8001	→	1000	0000	0000	0001
Reg$_{15}$	800F	→	1000	0000	0000	1111

c) Address range of RAM = 0000 ⟷ 07FF
ROM = 4000 ⟷ 4FFF
interface register = 8000 ⟷ 800F.

d) For 8 RAM, the decoder size is 3×8 (as $2^3 = 8$).
For 4 ROMs, the decoder size is 2×4 (as $2^2 = 4$).

Q3. A computer uses RAM chips of 1024 × 1 capacity.
a) How many chips are needed and how should their address lines be connected to provide a memory capacity of 1024 bytes.

b) How many chips are needed to provide a memory capacity of 16 k bytes. Explain in words how the chips are connected to the address bus.

Ans: The given size of RAM chip = 1024 × 1 bits
= 1024 bits.

a) According to the question, we have to achieve total of 1024 bytes
= 1024 × 8 bits.

Thus, required number of RAM chip = $\frac{1024 \times 8}{1024}$ = 8 chips

As $2^{10} = 1024$,

Thus, there will be 10 common address lines for all the RAM chips.

Total number of RAM chips = 8.

As $2^3 = 8$
Thus, 3×8 decoder is needed to connect

all the RAM.

CS_0 of RAM_0 is connected to 0th output of decoder.
CS_1 of RAM_1 is connected to 1th output of decoder.
and so on.

The decoder will use 11, 12 and 13 address line of CPU as input.

In order to get 8 bit data line all the rams are connected parallely to generate 8 bit data line (8 columns). All RA same chip select line.

b) In the question, we have to achieve 16 kbytes of memory

$= 16 \times 1024 \times 8$ bits.

Thus, RAM chip required $= \frac{16 \times 1024 \times 8}{1024} = 128$ chips.

Here, also address line 1 to 10 is common to all RAM address line.

As $2^7 = 128$,

the

To achive 8 data line, there will be 8 columns of RAM.

Thus, there will be $\frac{128}{8} = 16$ rows in each column.

To connect 16 rows, 4×16 decoder will be needed.

The CPU address lines of 11, 12, 13 and 14 will be the input to the decoder and 0 output of decoder will connect row 0 CS_1, 1 output of decoder to row 1 and so on.

block number

* Cache Memory → For faster memory reference operations CPU stores the most frequently used data of main memory in a very small memory, called cache memory.

Three very basic points to be remembered,

① Smallest partition of cache memory is line
main memory is block
virtual memory is page.

② Mapping of virtual to main memory and main memory to cache possible only when, Size of page = Size of block = Size of line.

③ The smallest data chunk is called word. The mapping is done while a complete block or page is mapped.

* Three different type of cache mapping for cache →

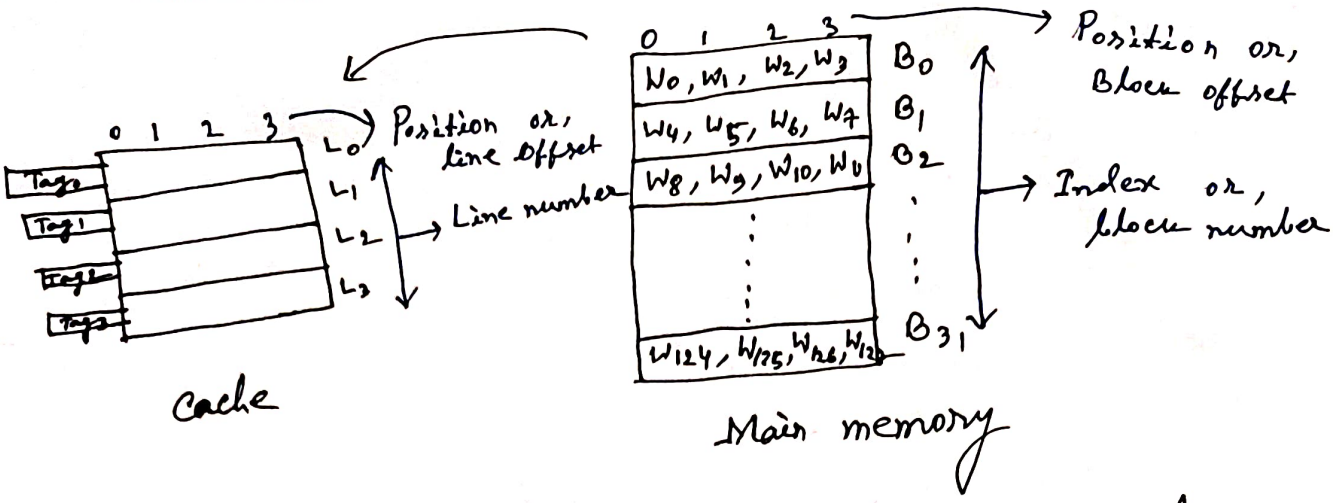
① Direct mapping → Each block of main memory has a fixed predefined line to be mapped in cache memory.

The process is,

$$k \bmod n$$

k ↙ Block number

n → Total number of lines in cache.



In the given example, Block size = 4 words.

Total Blocks = 32 blocks (0-31).

Total Lines = 4 lines (0-3).

Line size = 4 words.

Total Block = $32 = 2^{5}$ → 5 bit is required for index

Words each block = $4 = 2^{2}$ → 2 bit is required for block offset.

Thus, physical address of main memory

PA = | 5 | 2 |

5 → Index field (Block number) (s)

2 → Block offset. (w)

For cache memory, Words each line = $4 = 2^{2}$

→ Line offset or, Block offset.

Total lines = $4 = 2^{2}$ → Index (Line number)

Thus, address of cache memory, (For considered example)

| 3 | 2 | 2 |

Tag (s-r) | Index r | Block offset w

Thus, for Line 0, only B_0, B_4, B_8, B_{12}, B_{16}, B_{20}, B_{24} and B_{28} can be mapped as, mod 4 for all these blocks are 0.

Advantage → The CPU needs to check only one line for cache hit, the operation is fastest. High searching speed.

Disadvantage → The same preference blocks can not be mapped in cache, bad memory management.

(2) Associative mapping → Here, any block can be mapped anywhere or any line.

The address of cache, (For considered example)

Tag — Block offset.

Advantage → Good memory management as blocks can be mapped in any line.

Disadvantage → The whole cache needs to be checked to check for cache hit, low searching speed.

(3) k-way setassociative mapping → It is a hybrid of direct and associative mapping. cache memory is divided in sets and every block has a predefined block number like direct mapping and after determination of set the block can be mapped in any line under that set like associative mapping.

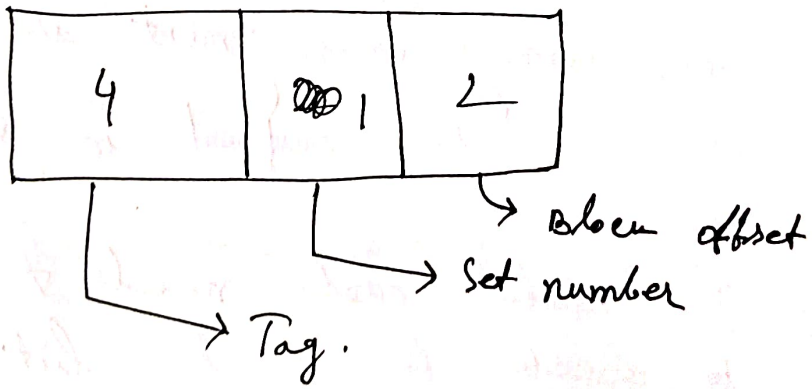
k-way

↓

Each set has k lines.

Example → For 2-way set associative, 2 lines in each set.

The address of cache memory, (For considered example)



The set is determined as, k mod n

k → Block number

n → Total set number.

Q.1. Suppose there is a direct cache mapping. The size of main memory is 128 kB and cache is 16 kB. The block size is 256 bytes. Find,

a) Number of bits in tag
b) Tag directory size.

Ans: The mapping is direct mapping.

Size of main memory = 128 kB
= 128 × 1024 bytes

Size of each block = 256 bytes.

Thus, total number of blocks = $\frac{128 \times 1024}{256} = 512$

Again size of main memory = 128 × 1024 bytes
$= 2^7 \times 2^{10}$
$= 2^{17}$ → 17 bits PA.

Thus, $PA_{main\ memory} = 17$ bits.

Total blocks = $512 = 2^9$ → 9 index field.

Thus, each block has $2^{(17-9)} = 2^8$ words. → 8 block offset.

$PA_{main\ memory}$ | 9 | 8 |

9 → Index
8 → Block offset

Thus. cache must have same block offset
= 8 bit.

Again, cache size = 16 kB
= 16 × 1024
8

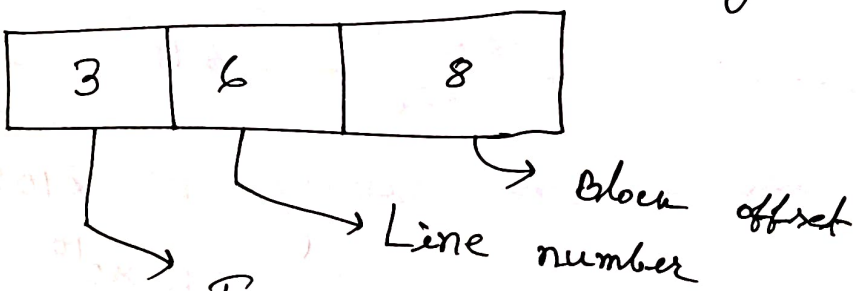
each line size = 2^8

Thus, total number of line = $\frac{16 \times 1024}{2^8}$

$= \frac{2^4 \times 2^{10}}{2^8}$

$= 2^6$ → Line number.

Thus, address of cache memory,



a) Thus. tag has 3 bits.

b) The tag directory size = Number of tags × Tag size
= $2^6 \times 3$
= 64 × 3 bits
= 24 bytes.

Q.2. Consider a 4-way set associative mapped cache memory. It has block size 4kB, the size of main memory is 16 GB and tag has 10 bits. Find,

a) Size of cache memory

b) Tag directory size.

Ans:

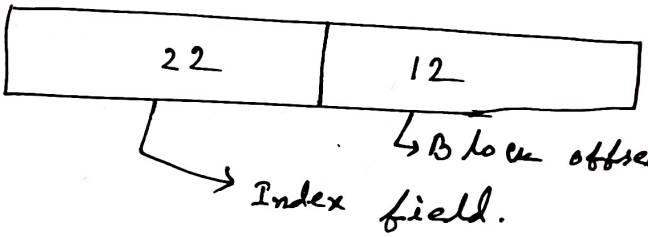
Main memory size = 16 GB
$= 16 \times 2^{10} \times 2^{10} \times 2^{10}$ bytes
$= 2^{34}$ bytes.

Block size = 4 KB
$= 4 \times 2^{10}$ bytes
$= 2^{12}$ bytes.

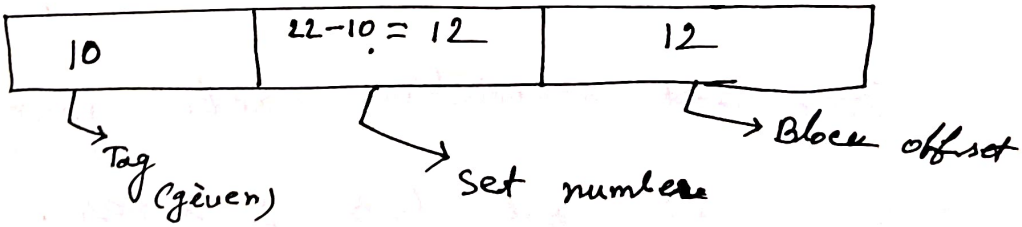
Thus, total block number = $\frac{2^{34}}{2^{12}} = 2^{22}$ → Index field.

Again block size = 2^{12} bytes → Block offset.

Thus, $PA_{\text{main memory}}$ =



Thus, address of cache memory,



Total set in cache memory = 2^{12} sets.

Its a 4-way set associative mapping.

Thus, each set has 4 lines.

Thus, total line = $2^{12} \times 4 = 2^{14}$ lines.

a) Size of cache = Tag size × Total lines
+ Total lines × Line size

$= \frac{10 \times 2^{14}}{8} + 2^{14} \times 2^{12}$

=

a) Size of cache = Line number × Line size

$= 2^{14} \times 4\,kB$

$= 2^4 \times 4 \times 2^{10}\,kB$

$= 64\,MB.$

b) Tag directory size = Tag size × Line number

$= 10 \times 2^{14}$ bits

$= 10 \times 2^{11}$ bytes

$= 20\,kB.$

Q.3. A 2-way set associative cache memory uses blocks of four words. The cache can accomodate 2048 words. The main memory size is 128k × 32. Formulate all partinent to construct the cache memory. What will be the size of cache memory?

Ans: 2-way set associative cache memory.

Thus, each set has 2 lines.

Each block has 4 words,

Thus, each line has 4 words.

cache has total 2048 words.

$$\text{Total lines} = \frac{2048}{4} = 512 \text{ lines.}$$

Thus, total set in cache $= \frac{512}{2} = 256$ sets.

Now, $256 = 2^8$, 8 bits is required to represent set number.

Each line has 4 words $= 2^2$, 2 bits are required to represent word position or, block offset.

~~Thus, address of cache is.~~

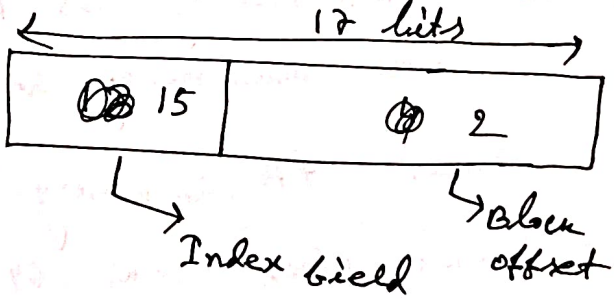
Size of main memory $= 128\text{K} \times 32$

(128K: Words; 32: Words size.)

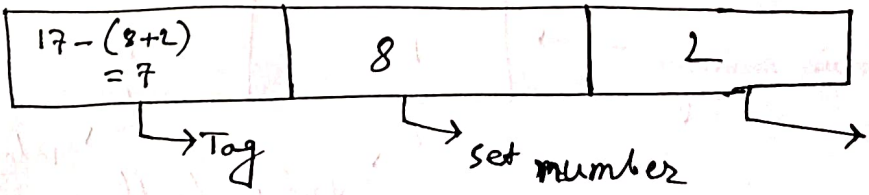
Thus, main memory has, 128×1024 Words

$:= 2^{17}$ Words.

Thus, $PA_{\text{main memory}}$



Thus, address of cache is,



The size of cache = Tag directory + Line number × Size of each

= Total line number × Tag size + Line number × Data in each line.

= 512 × 7 + 512 × 4 × 32

= 512 × (7 + 128)

= 512 × (135)

= 69,120 bits

= 8640 bytes.

Q.4. Consider a cache memory consisting of 64 blocks of 64 words each. Also assume that the main memory is addressable by a 16 bit address and accomodates 12-bit data. How many bits exists in the tag, index, Block and word field for the 4-way set associative cache memory mapping? Include a valid bit. What is the size of the cache memory?

Ans:

Main memory has 16 bit address line and 12 bit data line.

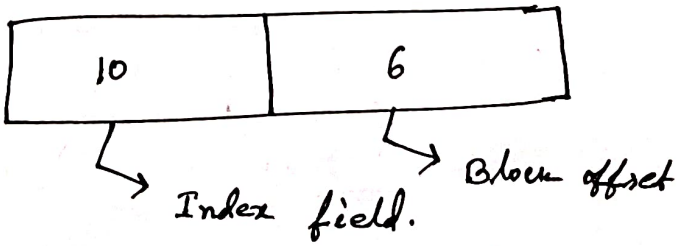
Thus, main memory has 2^{16} words having each size 12 bits.

Thus, $PA_{main\ memory}$ = 16 bit.

Each block has 64 words = 2^6

Thus, 6 bit block offset.

Thus, $PA_{main\ memory}$ →



Again, it is a 4-way set associative mapping.

Thus, each set has 4 lines.

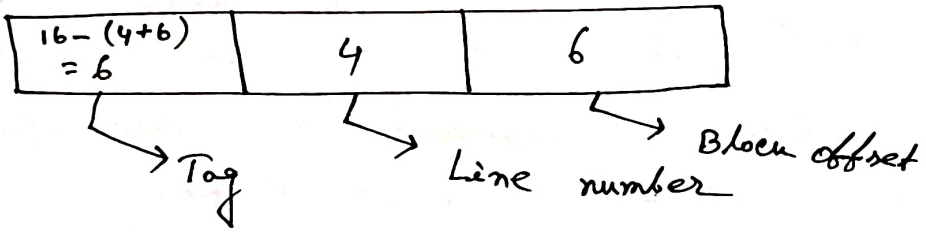
Total blocks or lines in cache is, 64 blocks.

Thus, total set $= \frac{64}{4} = 16$ sets.

Now, $16 = 2^4$.

Thus 4 bits are needed to represent set number.

Thus, address of cache is,



Thus, in cache, Tag = 6 bits

Line number or index = 4 bits

and word or block offset = 6 bits.

Including a valid bit, the size of each line

= Tag + valid bit + size of 64 words

= 6 + 1 + 64 × 12

= 7 + 768

= 775 bits.

Thus, total size of cache memory = Total line × Size of each line

= 64 × 775 bits

= $\frac{64}{8}$ × 775 bytes

= 775 × 8 bytes = $\frac{775 \times 8}{1024}$ KB = 6.05 KB.

Q.5. Consider a cache of 256 blocks of 16 words each. Assume that the main memory is addressable by a 16 bit address and 12 bit data lines. How many bits are there in tag, index, block and word field for a 2-way set associative cache mapping? Include a valid bit. What is the size of cache memory?

Ans: Main memory has 16 bits address and 12 bits data lines.

Thus, 2^{16} words each of size 12 bits in main memory.

Each block has 16 words.

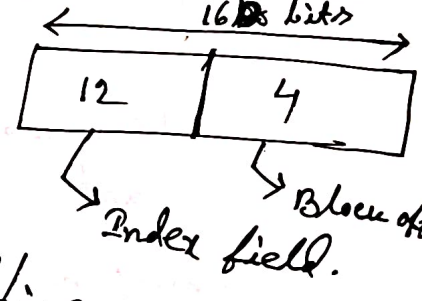
Thus, total $\frac{2^{16}}{16} = \frac{2^{16}}{2^4} = 2^{12}$ blocks in main memory.

12 bits are required in index field.

Again, each block has $16 = 2^4$ words.

Thus, 4 bits are required for block offset.

Thus, physical address of main memory =



It is a 2-Way set associative mapping.

Thus, each set has 2 lines.

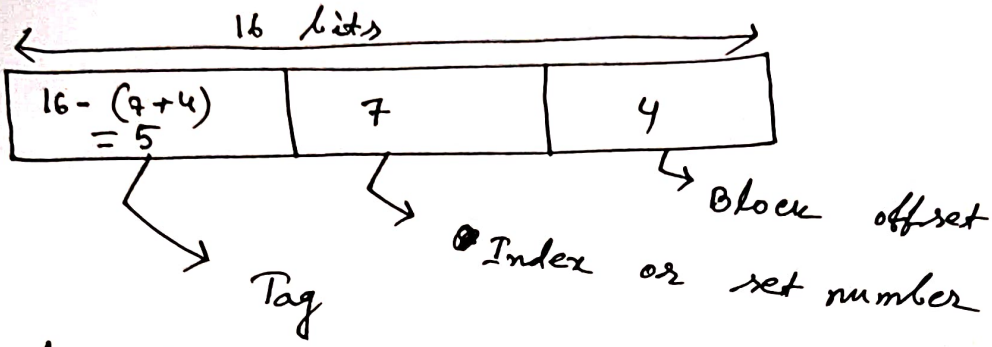
Total 256 lines or blocks in cache.

Thus, $\frac{256}{2} = 128$ sets are there.

Now, $128 = 2^7$, 7 bits are required to represent set number or, index for cache memory.

Thus, address of cache is

Thus, address of cache is,



Thus, cache has, Tag = 5 bits

Index = 7 bits

and block offset = 4 bits.

Including a valid bit, the size of cache

= Total line × size of each line

= 256 × (Tag + Valid bit + Data of each line)

= 256 × (Tag + Valid bit + Total words × Word size)

= 256 × (5 + 1 + 16 × 12)

= 256 × (6 + 192)

= 256 × 198 bits

$= \frac{256 \times 198}{8 \times 1024}$ KB

≈ 6.2 KB.

Q.6. Consider cache hit ratio is 0.7, cache access time 70ns and memory access time is 700 ns. What will be the average access time of the system?

Ans: Given $T_c = 70$ ns

$T_m = 700$ ns.

Hit ratio = 0.7.

Thus, miss ratio = 1 - Hit ratio = 1 - 0.7 = 0.3

Now, average access time = Tc × Cache hit ratio + (Tc + Tm) × miss ratio

= 70 × 0.7 + (70 + 700) × 0.3

= 49 + 231.0

= 280 ns.

Q.7. The access time of a cache memory is 100 ns and that of main memory is 1000 ns. It is seen that 80% of memory request is read and rest are write. The hit ratio for read is 0.9. A write through process is used. What will be the access time for only memory read? What will be the average access time for both read and write operation? What is the hit ratio considering read request?

Ans: Given, $T_c = 100$ ns

$T_m = 1000$ ns.

80% Read and (100 - 80) = 20% write request.

Hit ratio for read = 0.9

Thus, average access time = $H \times T_c + (1-H) \times (T_c + T_m)$

= $0.9 \times 100 + (1 - 0.9)(100 + 1000)$

= 90 + 110

= 200 ns.

Average access time for read and write access,

= $\underbrace{0.8 \times 200}_{\text{Read}} + \underbrace{0.2 \times 1000}_{\text{Write}}$

= 160 + 200

= 360 ns.

Hit ratio for read = Probability that the request is read × hit ratio of read.

$\equiv \frac{80}{100} \times 0.9$

$\equiv 0.72$.

GLA UNIVERSITY
MATHURA

COMPUTER ORGANIZATION
MODULE-II

(Input-Output Organization & Data Transfer)

RAM Organization:

RAM memory can be organized by 2 ways:

a. Two dimensional RAM (2D RAM)

b. 2 ½ dimensional RAM (2.5D RAM)

Two dimensional RAM (2D RAM):

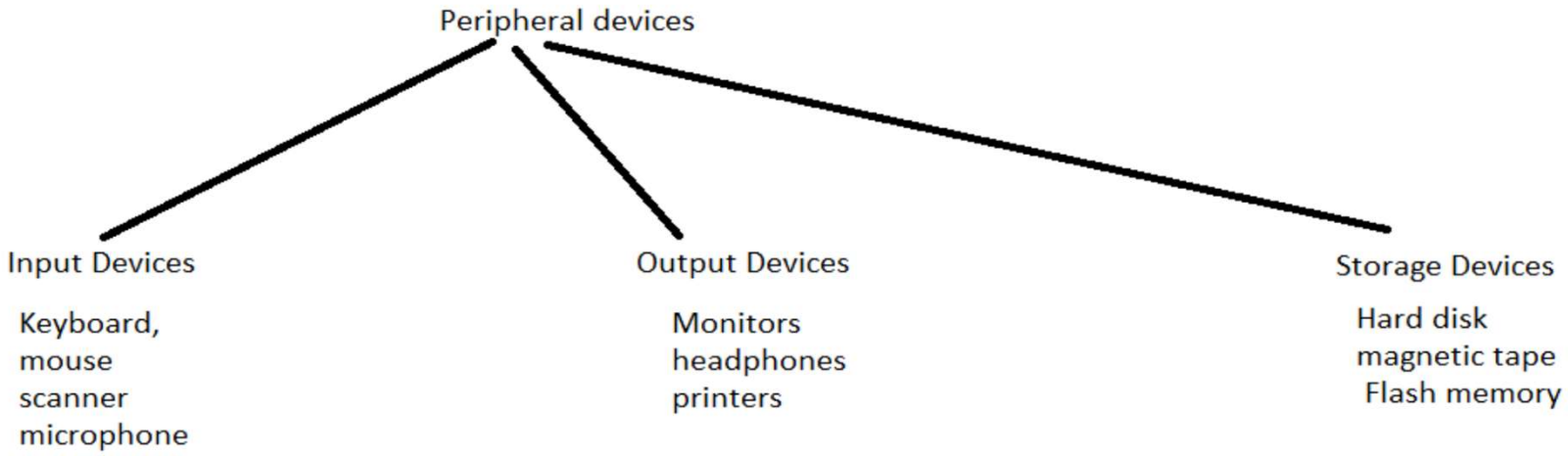
- ✓ In 2D RAM hardware is fixed.
- ✓ It requires more number of gates in decoder logic.
- ✓ It is more complex.
- ✓ It is difficult to fabricate and design.

2 ½ dimensional RAM (2.5D RAM):

- ✓ In 2.5D RAM hardware can be changed.
- ✓ It requires less number of gates in decoder logic.
- ✓ It is less complex.
- ✓ It is easy to fabricate and design.

Input-Output Organization:

Peripheral Devices: A Peripheral Device is defined as the device which provides input/output functions for a computer.
It is generally classified into 3 basic categories:



- Peripherals that provide auxiliary storage are magnetic tape and magnetic disk.
- Input and output devices communicate alphanumeric information by using ASCII 7bit code.

Advantage of Peripherals Devices:

- It is helpful for taking input very easily.
- It is also provided a specific output.
- It has a storage device for storing information or data
- It also improves the efficiency of the system.

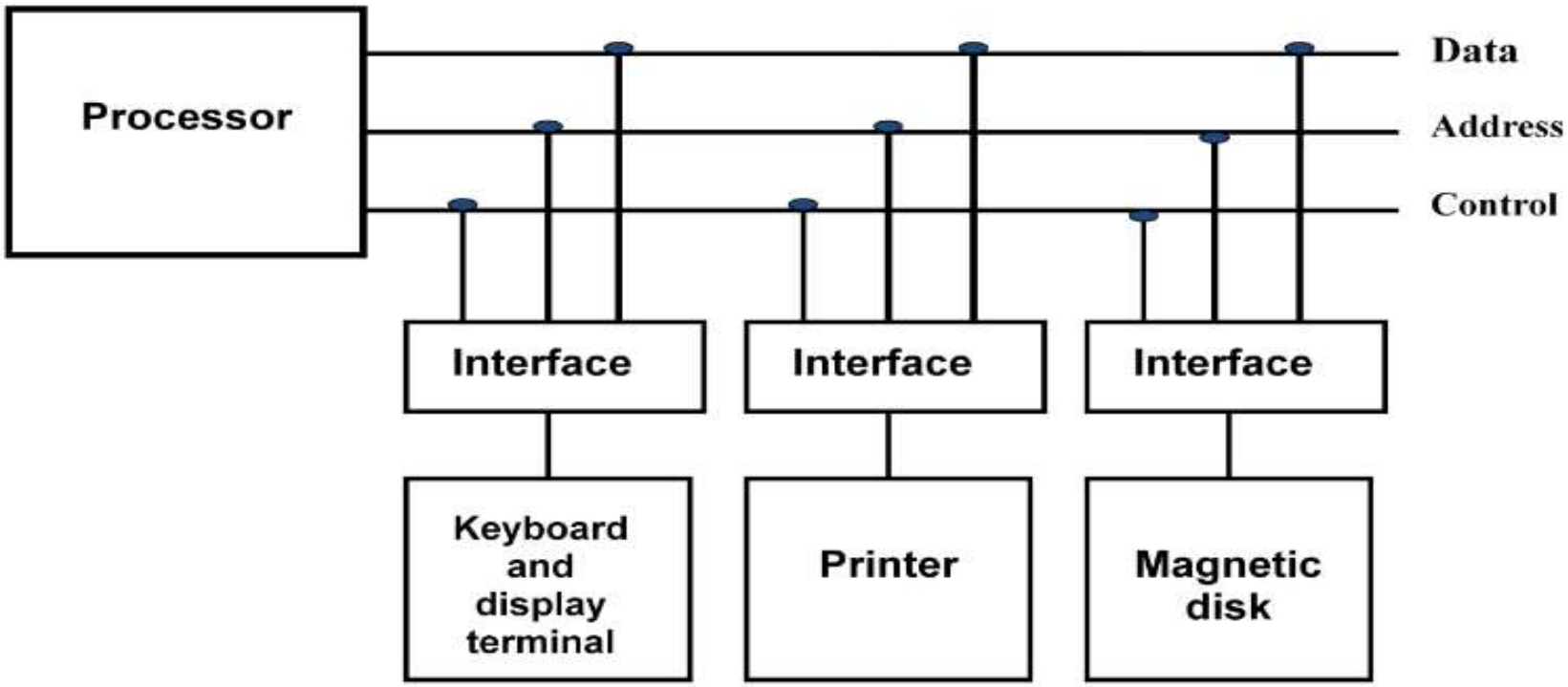
Input-Output Interface:

- ✓ I/O interface provides a method for transferring information between internal storage and external I/O devices.

- ✓ The purpose of interfacing are as follows:
  - Peripherals are electromechanical & electromagnetic devices and are interacting with electronics devices (CPU). Therefore, a conversion of signal values may be required.
  - Data transfer rate of peripherals is usually slower than transfer rate of CPU.
  - Data codes and formats in peripherals differ from word format in CPU and memory.
  - Operating modes of peripherals are different from each other and each one must be controlled without disturbing other peripherals connected to the CPU.
- ✓ To resolve these differences, computer system includes Interface units between CPU and peripherals to supervise and synchronize all input and output transfers.

I/O Bus and Interface Modules:

- ✓ Each peripheral has its own controller that operates a particular electromechanical device.
- ✓ To communicate with a particular device, the processor places a device address on the address lines.



Connection of I/O bus to input-output devices

I/O Bus and Interface Modules:

- When the interface detects it own address, it activates path between bus lines and the device.

- At the time address is made available in address lines, the processor provides function code (I/O command) in the control lines.

Types of I/O command:

- Control command: To activate and inform what to do.
- Status command: To test various status conditions.
- Data Output data: It causes the transfer of data from bus into one of its registers.
- Data Input Command: Interface receives data from peripheral and places them on buffer register where it is put into data lines.

I/O versus Memory Bus:

- In addition to communicating to I/O, processor must also communicate with memory unit.

- There are three ways that computer buses can be used to communicate with memory and I/O:

  - Use two separate buses, one for memory and one for I/O. (IOP)
  - Use one common bus for both memory and I/O but have **separate control lines** for each. (Isolated I/O)
  - Use one common bus for memory and I/O with **common control lines.** (Memory Mapped I/O)

Isolated I/O:

- The distinction between memory and I/O transfer is made through separate read and control lines.
- I/O read and I/O write are enabled during I/O transfer and Memory read/write are enabled during memory transfer.
- In the isolated I/O configuration, CPU have distinct input and output instructions where each of it will be associated with address of the interface register.
- When the CPU fetches and decodes the I/O instruction, it places the address associated with the instruction on the common address lines and enables I/O read or I/O write control line.
- When the CPU fetches and decodes the Memory instruction, it places the address associated with the instruction on the common address lines and enables Memory read or Memory write control line.
- The isolated I/O method isolates memory and I/O addresses.

Memory Mapped I/O:

- Here the same address space is used for both memory and I/O. The computer treats interface register as a part of memory system.

- The assigned address cannot be used for storing memory words, which reduces memory address range available.

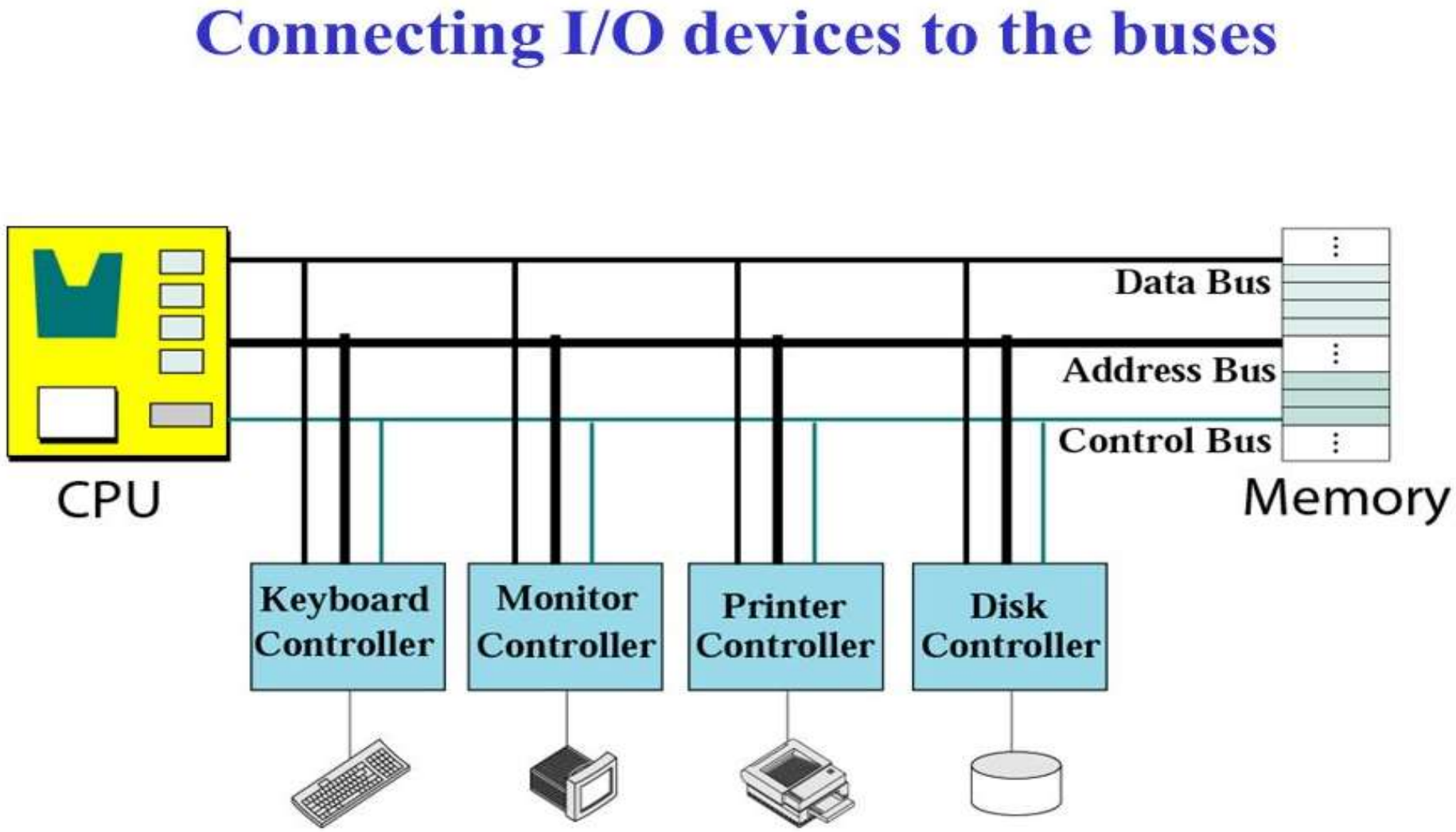
- In a memory-mapped I/O organization, there are no specific, input or output instruction.

- CPU manipulate I/O data residing in interface registers with the same instruction used to manipulate memory words.

- Load and store instruction used for reading/writing from memory and can be use for input or output data from I/O instruction.

- With memory-mapped I/O all instructions that refer to memory are also available for I/O.

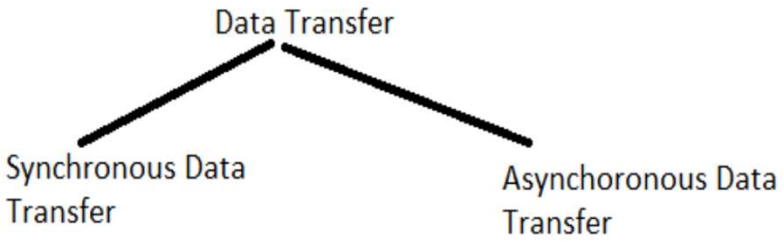
Memory Mapped I/O:

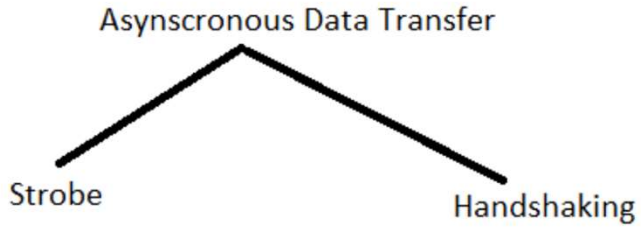


Data Transfer:

- If the registers in the interface share a common clock with the CPU registers, the transfer is **synchronous**.
- **Asynchronous** data transfer requires that control signals be transmitted between communicating units to indicate the time at which data is being transmitted.
- Asynchronous data transfer can be accomplished by **Strobe & Handshaking**.

S. No.	Synchronous transmission	Asynchronous transmission
1.	Transmitter and receivers are synchronized by clock.	Transmitter and receivers are not synchronized by clock.
2.	Data bits are transmitted with synchronization of clock.	Bits of data are transmitted at constant rate.
3.	Data transfer takes place in blocks.	Data transfer is character oriented.



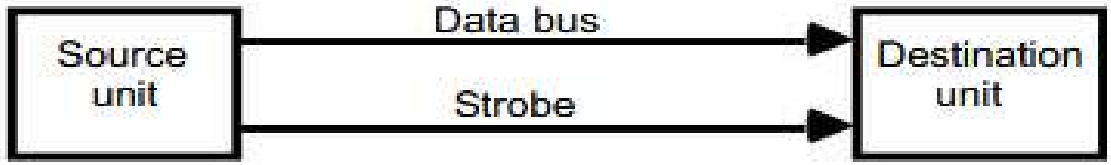


Strobe Control:

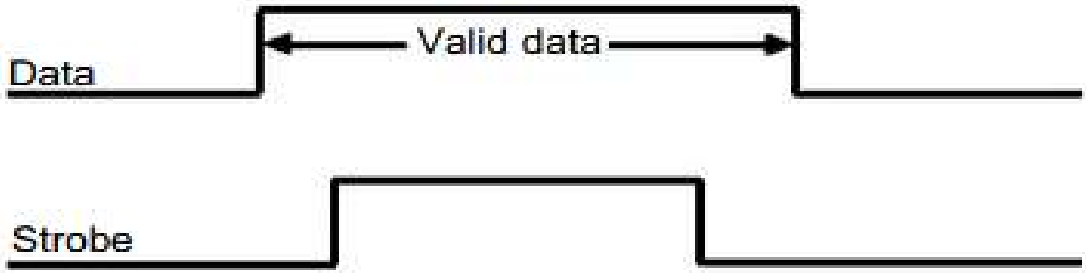
Strobe control:

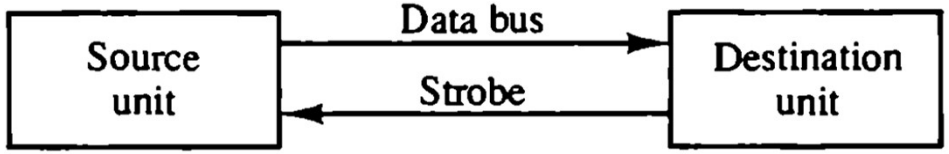
- It employs single control line.
- It can be activated either by source or destination unit.

Block Diagram



Timing Diagram





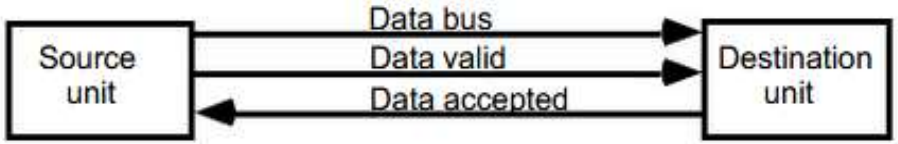
Handshaking:

- ✓ The disadvantage of the strobe method is that the source unit that initiates the transfer has no way of knowing whether the destination unit has actually received the data item that was placed in the bus.
- ✓ The handshake method solves this problem by introducing a second control signal that provides a reply to the unit that initiates the transfer.
- ✓ The basic principle of the two-wire handshaking method of data transfer is as follows.
  - ❖ One control line is in the same direction as the data flow in the bus from the source to the destination. It is used by the source unit to inform the destination unit whether there are valid data in the bus.
  - ❖ The other control line is in the other direction from the destination to the source. It is used by the destination unit to inform the source whether it can accept data.
  - ✓ The sequence of control during the transfer depends on the unit that initiates the transfer.

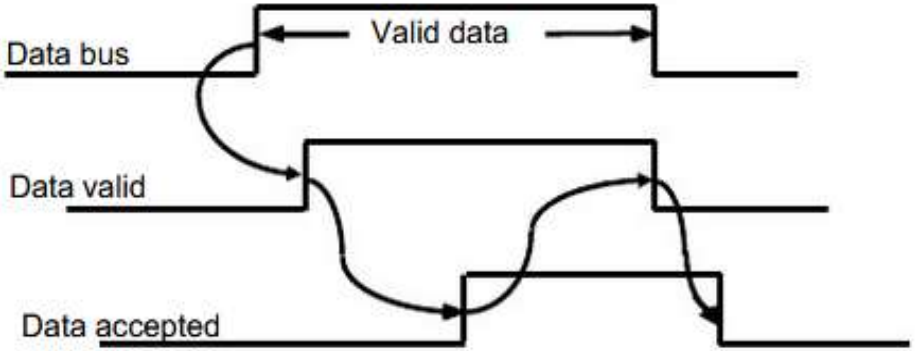
Handshaking:

The two handshaking lines are data valid, which is generated by the source unit, and data accepted, generated by the destination unit.

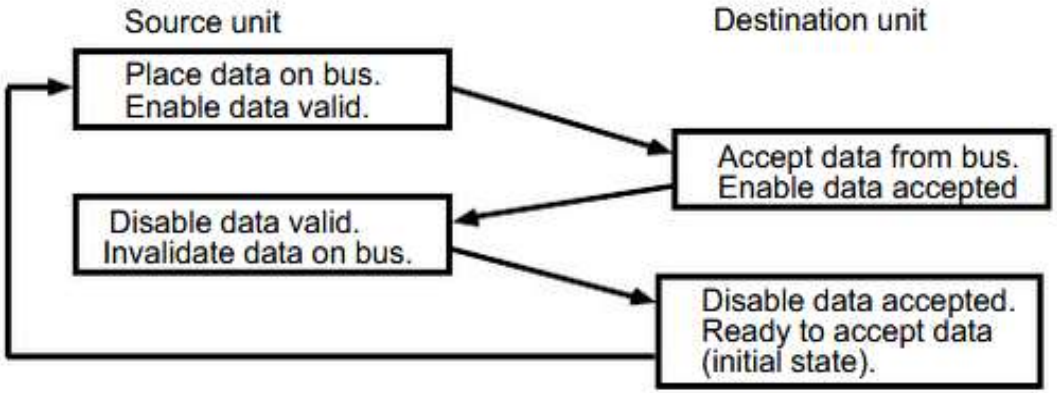
Block Diagram



Timing Diagram

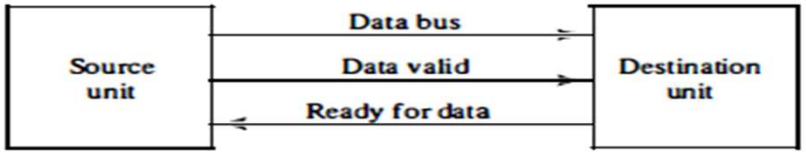


Sequence of Events

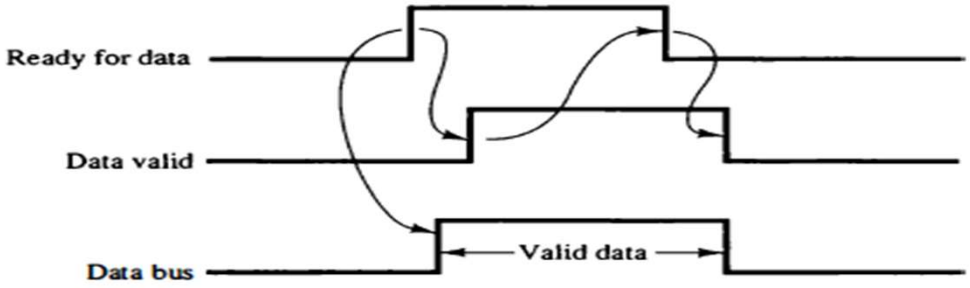


Handshaking:

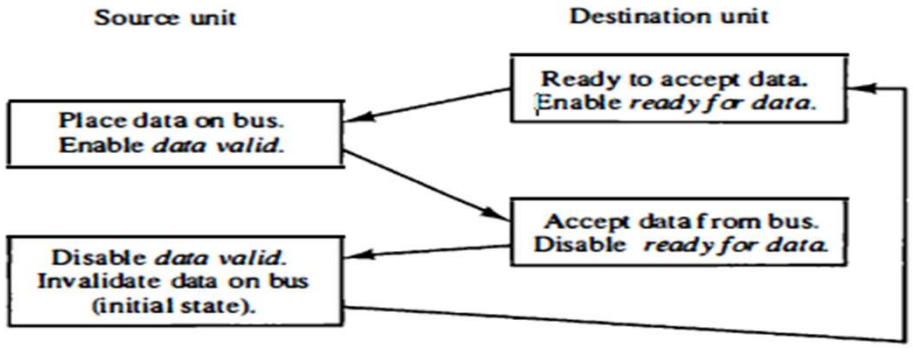
- In the destination-initiated transfer the name of the signal generated by the destination unit has been changed to ready for data to reflect its new meaning.
- The source unit in this case does not place data on the bus until after it receives the ready for data signal from the destination unit.



(a) Block diagram

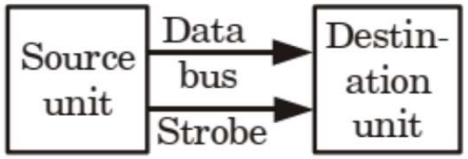
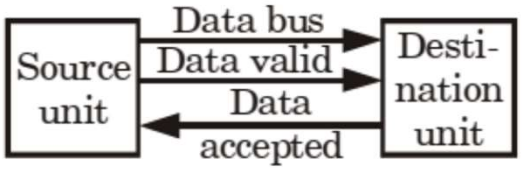
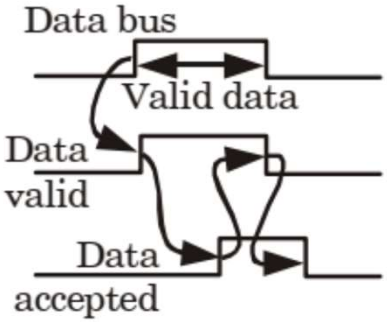


(b) Timing diagram



(c) Sequence of events

Difference Between Strobe & Handshaking Control:

S. No.	Parameter	Strobe control	Handshaking
1.	Control line	It employs a single control line to time each transfer.	It employs more than single control line to time each transfer.
2.	Acknowledgement	Reply message is not present.	Reply message is present.
3.	Block diagram		
4.	Timing diagram		

Modes of Transfer:

Data transfer between the CPU and the I/O devices may be handled in variety of modes. Some modes use the CPU as an intermediate path and others transfer the data directly to and from the memory unit.

Data transfer to and from peripherals may be handled in three ways:

- Programmed I/O
- Interrupt-initiated I/O
- Direct Memory Access (DMA)

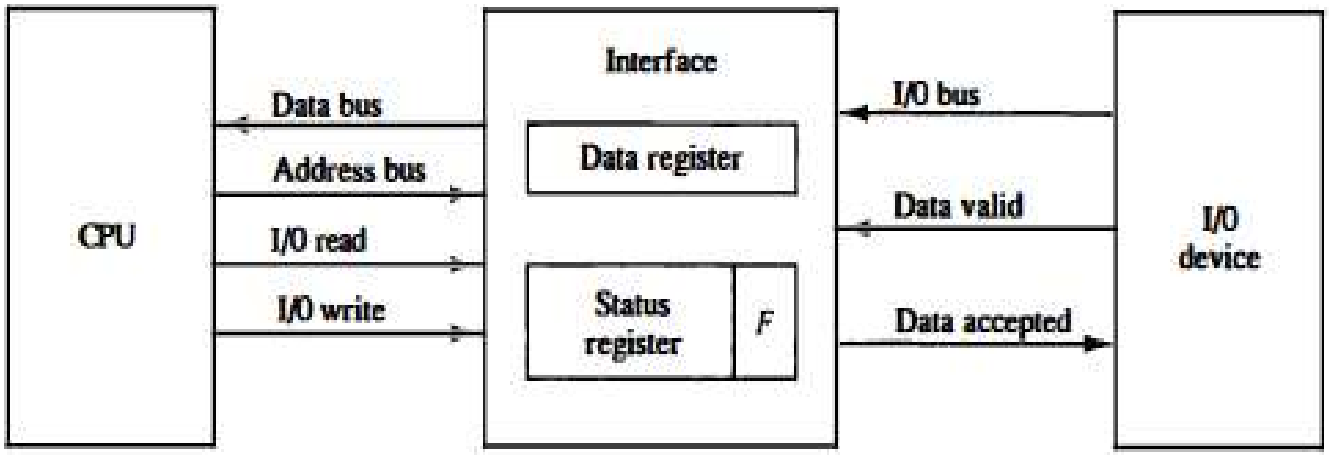
Programmed I/O:

- Programmed I/O operations are the result of I/O instructions. Each data item transfer is initiate by an instruction in the program.

- Usually the transfer is to and from a CPU register and peripheral. Other instructions are needed to transfer data between memory and CPU.

- Once a data transfer is initiated, the CPU is required to monitor the interface to see when a transfer can again be made.

Figure 11-10 Data transfer from I/O device to CPU.



F = Flag bit

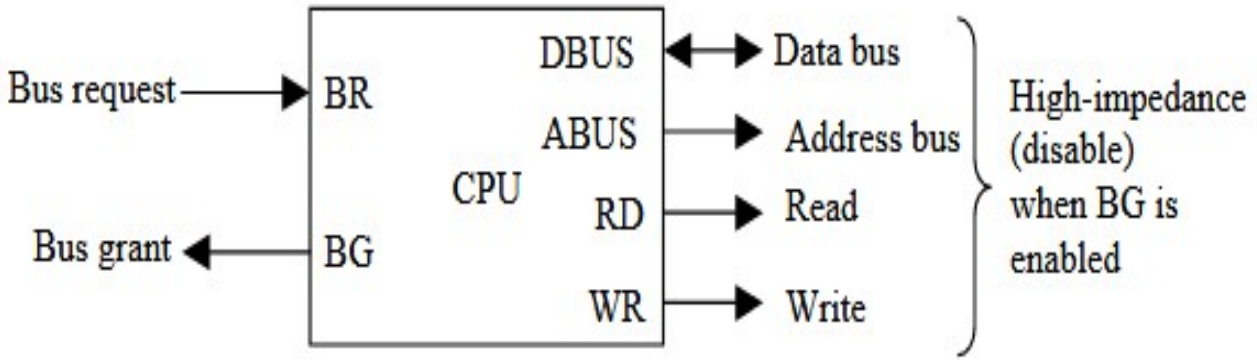
Interrupt Initiated I/O:

- In the programmed I/O CPU stays in program loop until the I/O indicates that it is ready for data transfer.
- This is time consuming process since it makes CPU busy needlessly.
- This can be avoided by using an interrupt facility.
- When the interface determines that device is ready for data transfer, it generates an interrupt request.
- Upon detecting external interrupt signal, the CPU momentarily stops the task it is processing.
- It then branches to fulfill the I/O request and return to the original task.
- Based on the concept of Priority Interrupt.

Direct Memory Access (DMA):

- The transfer of data between a fast storage device such as magnetic disk and memory often limited to the speed of CPU.
- Removing the CPU and letting the peripheral device manage the memory bus directly improve speed of transfer.
- Such transfer technique is called Direct Memory Access (DMA).
- A DMA controller takes over the buses to manage the transfer directly between I/O device and memory.
- During DMA transfer, the CPU is idle and has no control over memory buses.
- By using Bus Request (BR) and Bus Grant (BG) the buses are released to DMA controller.

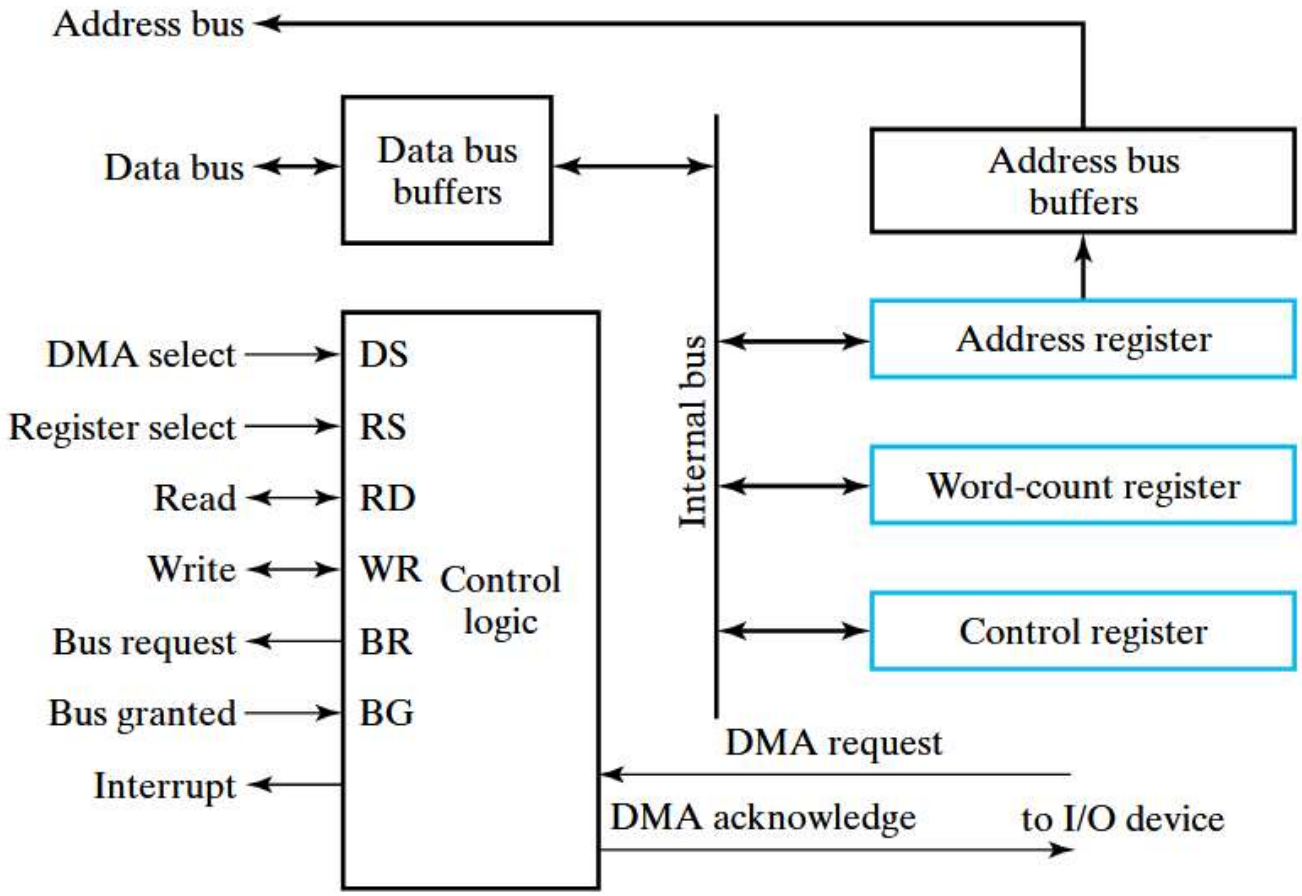
Figure 6.13 CPU bus signals for DMA transfer.



Direct Memory Access (DMA):

Data transfer ways:

- **Burst Transfer:** Here number of words are transferred in a block. Example: Magnetic disk.
- **Cycle stealing:** Allows the DMA controller to transfer one data word at a time after it must return the control of buses to CPU.

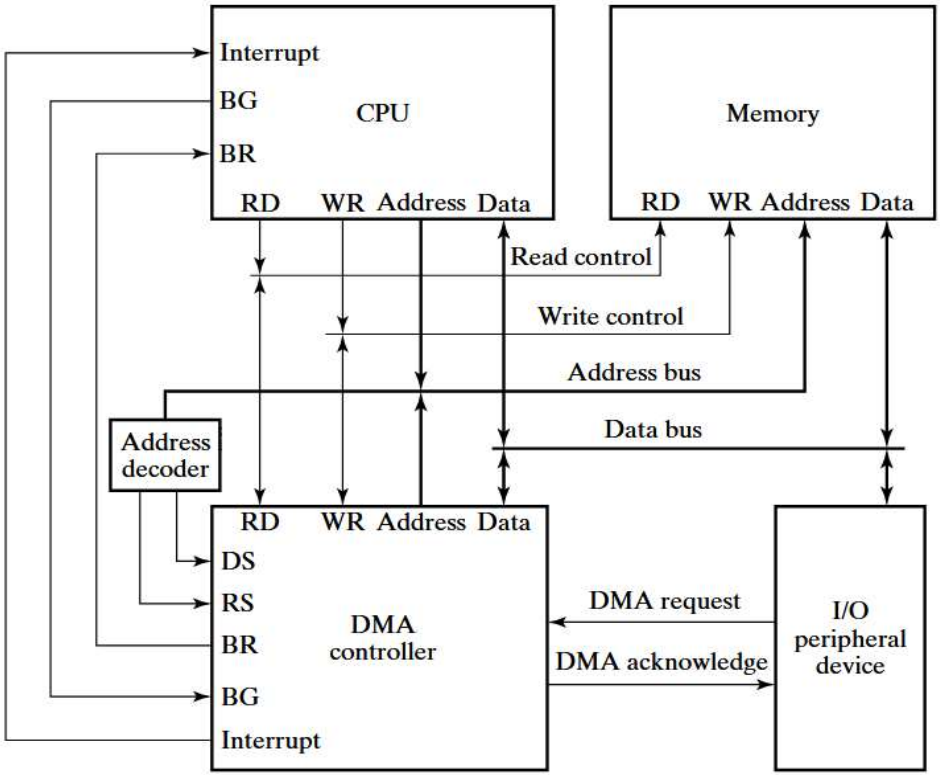


Initialization of DMA:

The CPU initializes the DMA by sending the following information through the data bus.

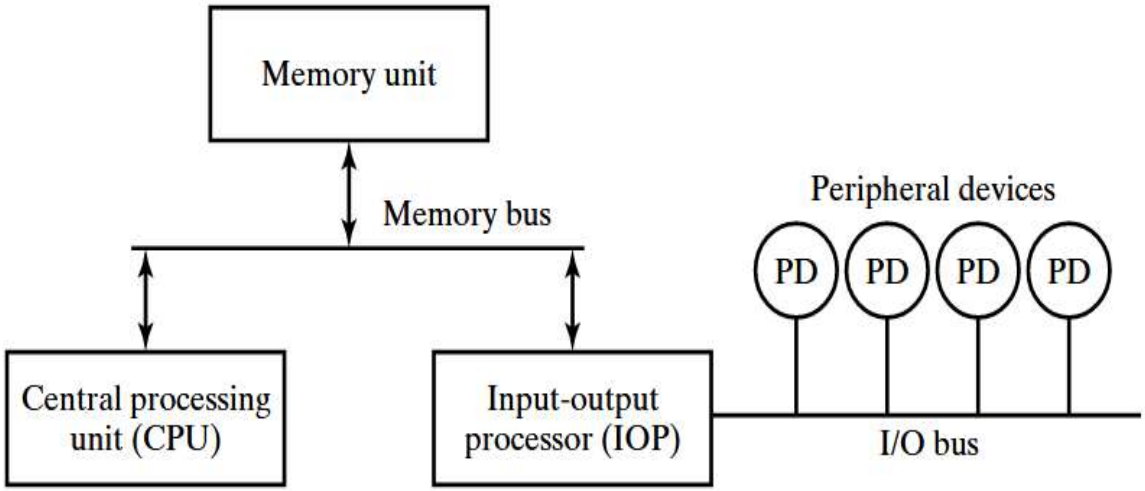
- The starting address of the memory block where data are available (for read) or where data are to be stored (for write).
- The word count, which is the number of words in the memory block.
- Control to specify the mode of transfer such as read or write.
- A control to start the DMA transfer.

DMA Transfer in a computer system:



Input-Output Transfer (IOP):

- An IOP takes care of input and output tasks.
- IOP is similar to CPU except IOP can fetch and execute an I/O instruction.
- CPU is usually assigned task of initiating the I/O program and thus IOP.
- The CPU is assigned the task of initiating all operations, but I/O instructions are executed in IOP.
- When an I/O operation is required, the CPU informs the IOP where to find the I/O program and then leaves the transfer details to the IOP.
- CPU instructions also test I/O status conditions needed for making decisions on various I/O activities.
- IOP is also responsible of taking care of data synchronization, formats etc between CPU and I/O devices.
- In most computers CPU is master and IOP is slave.
- The IOP typically asks for CPU attention by means of Interrupt.



Types of Interrupt:

- An **interrupt** is a signal to the processor emitted by hardware or software indicating an event that needs immediate attention.
- It alerts the processor to a high priority process requiring interruption of the current working process.
- In I/O devices one of the bus control lines is dedicated for this purpose and is called the ***Interrupt Service Routine*** *(ISR)*.

TYPES OF INTERRUPTS

There are many type of interrupts but basic type of interrupts are –

1. Hardware and software interrupts
2. Vectored and Non- vectored Interrupts
3. Mask able and non-maskable interrupts

1. Hardware and software interrupts

Hardware Interrupts: If the signal for the processor is generated from external device or hardware is called hardware interrupts. There are 5 Hardware Interrupts in processor. They are

1. INTR (Lowest Priority)
2. RST 7.5
3. RST 6.5
4. RST 5.5
5. TRAP (Highest Priority)

Software Interrupts are those which are inserted in between the program which means these are mnemonics of microprocessor.

There are 8 software interrupts: RST 0, RST 1, RST 2, RST 3, RST 4, RST 5, RST 6, RST 7.

Types of Interrupt:

2. Vectored and Non- vectored Interrupts

Vectored Interrupts are those which have fixed vector address (starting address of sub-routine) and after executing these, program control is transferred to that address. When an interrupt is occurred and program control automatically branches the program execution to a specific address.

Non-Vectored Interrupts (Scalar Interrupt) are those in which vector address is not predefined. Interrupts that have a variable address. When an interrupt device have to provide an address from where the execution of program will begin. **INTR** is the only non-vectored interrupt

3. Mask able and Non-maskable interrupts

Maskable Interrupt: The hardware interrupt that can be ignored or delayed for some time if the processor is executing a program with higher priority are termed as maskable interrupts**.** These interrupts are either edge-triggered or level-triggered, so they can be disabled. INTR, RST 7.5, RST 6.5, RST 5.5 are maskable interrupts.

Non Maskable Interrupt: The hardware interrupts that can neither be ignored nor delayed and must immediately be serviced by the processor are termed as non-maskable interrupts**.** It consists of both level as well as edge triggering and is used in critical power failure conditions. **TRAP** is a non-maskable interrupt.

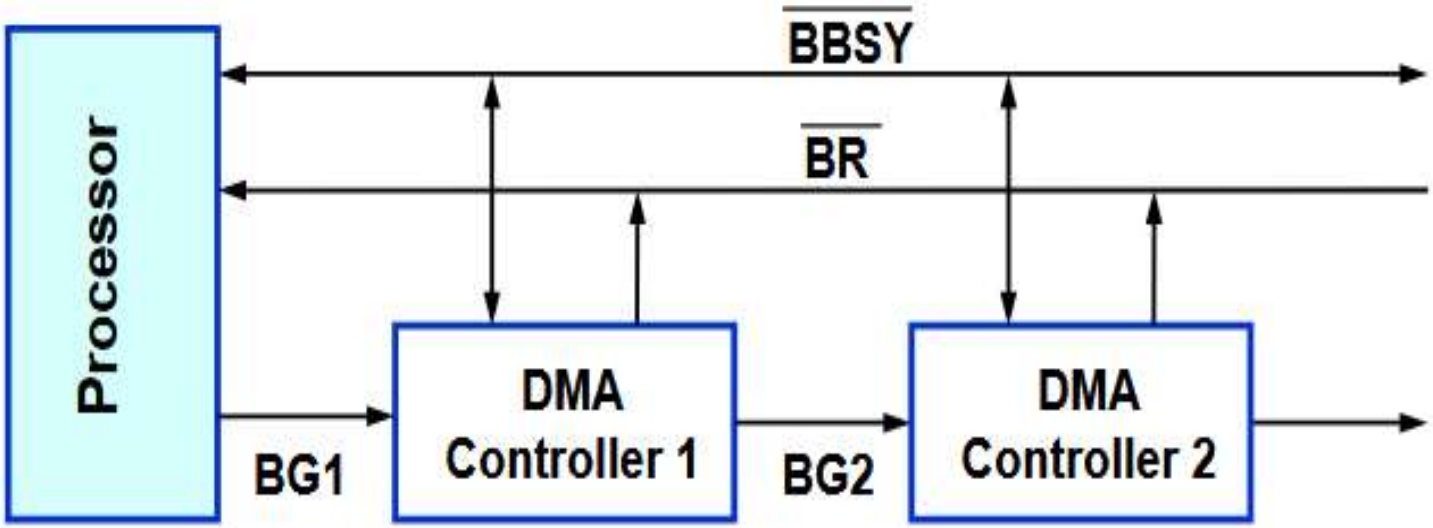
Types of Interrupt:

Major types of Interrupt are:

- **External Interrupt:** It comes from I/O devices, from timing device, or from any other external source.
- **Internal Interrupt:** It includes register overflow, invalid operation code, stack overflow etc.
- **Software Interrupt or Hardware Interrupt**: External and Internal interrupts are initiated from signals that occur in the hardware of the CPU. A software interrupt is initiated by executing an instruction.
- **Priority Interrupt**: In case several sources will request service simultaneously, in this case system must decide which device to service first. For ex: Polling, Daisy Chain.

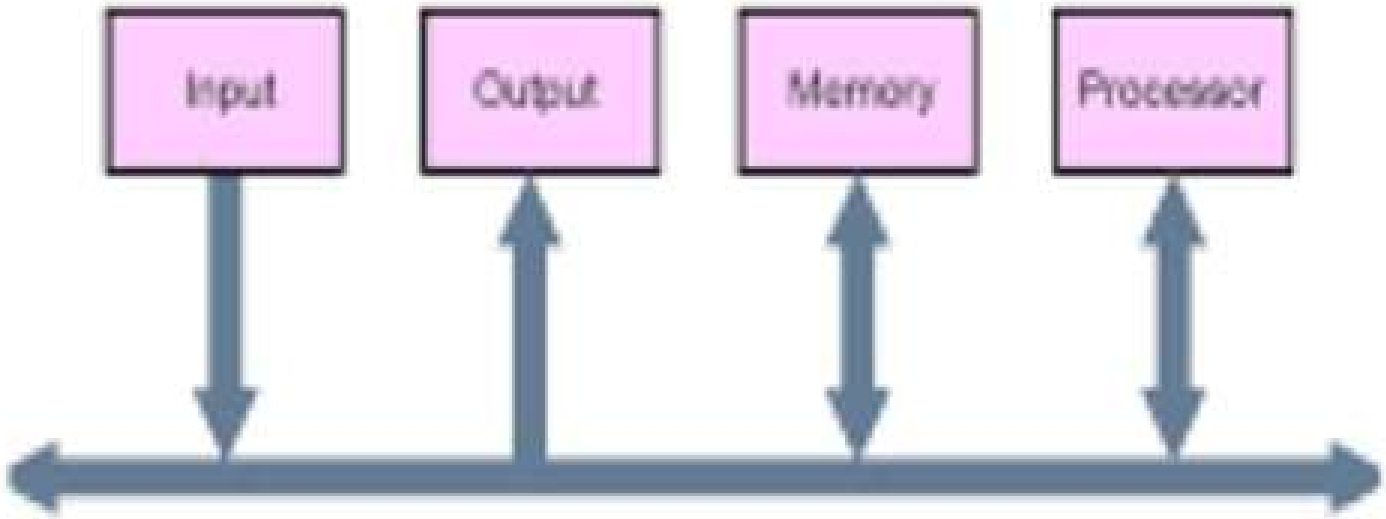
Bus Arbitration:

- The device that is allowed to initiate data transfers on the bus at any given time is called Bus master.
- Bus arbitration is the process by which the next device becomes Bus master and will do the data transfer.
- Two approaches: Centralized Arbitration and Distributed Arbitration.



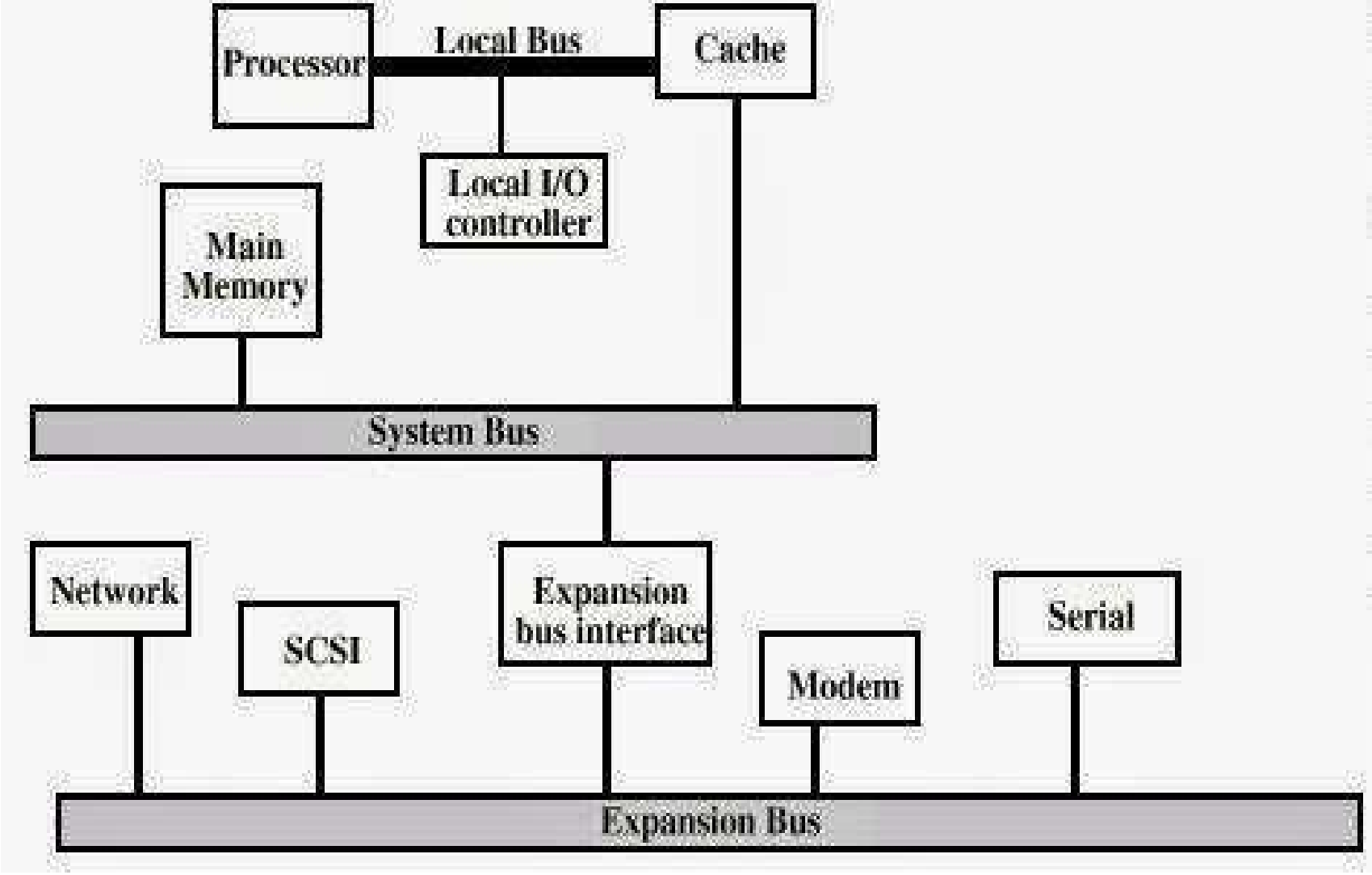
Buses:

- **Data Bus:** Bi-directional and transfers data.
- **Address Bus:** Uni-directional and sends the address.
- **Control Bus:** R/W, BR,BG etc.
- **Bus Structure**: Single bus



Buses:

- **Bus Structure:** Multi bus



THANK YOU